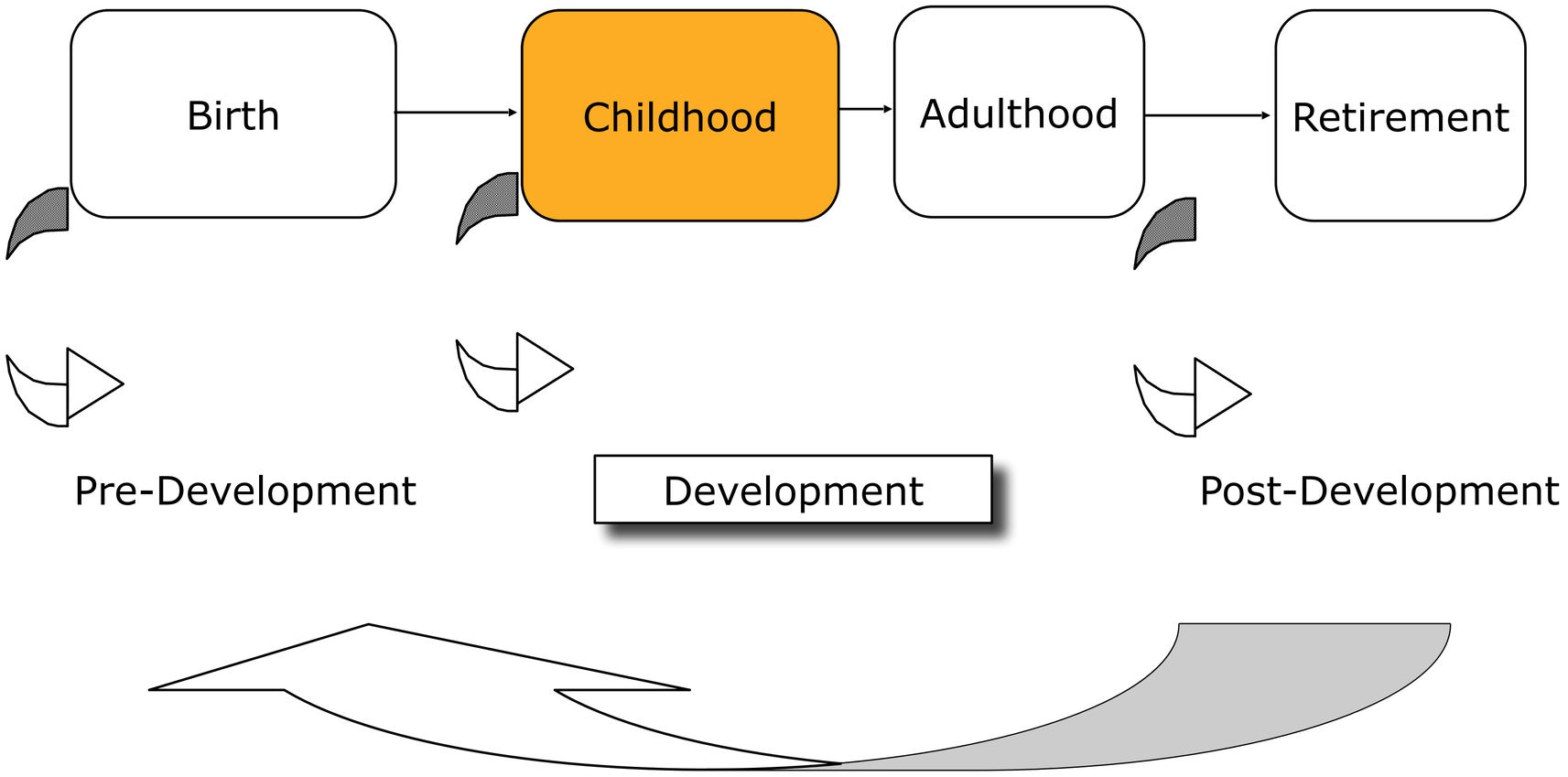
Software Dependability

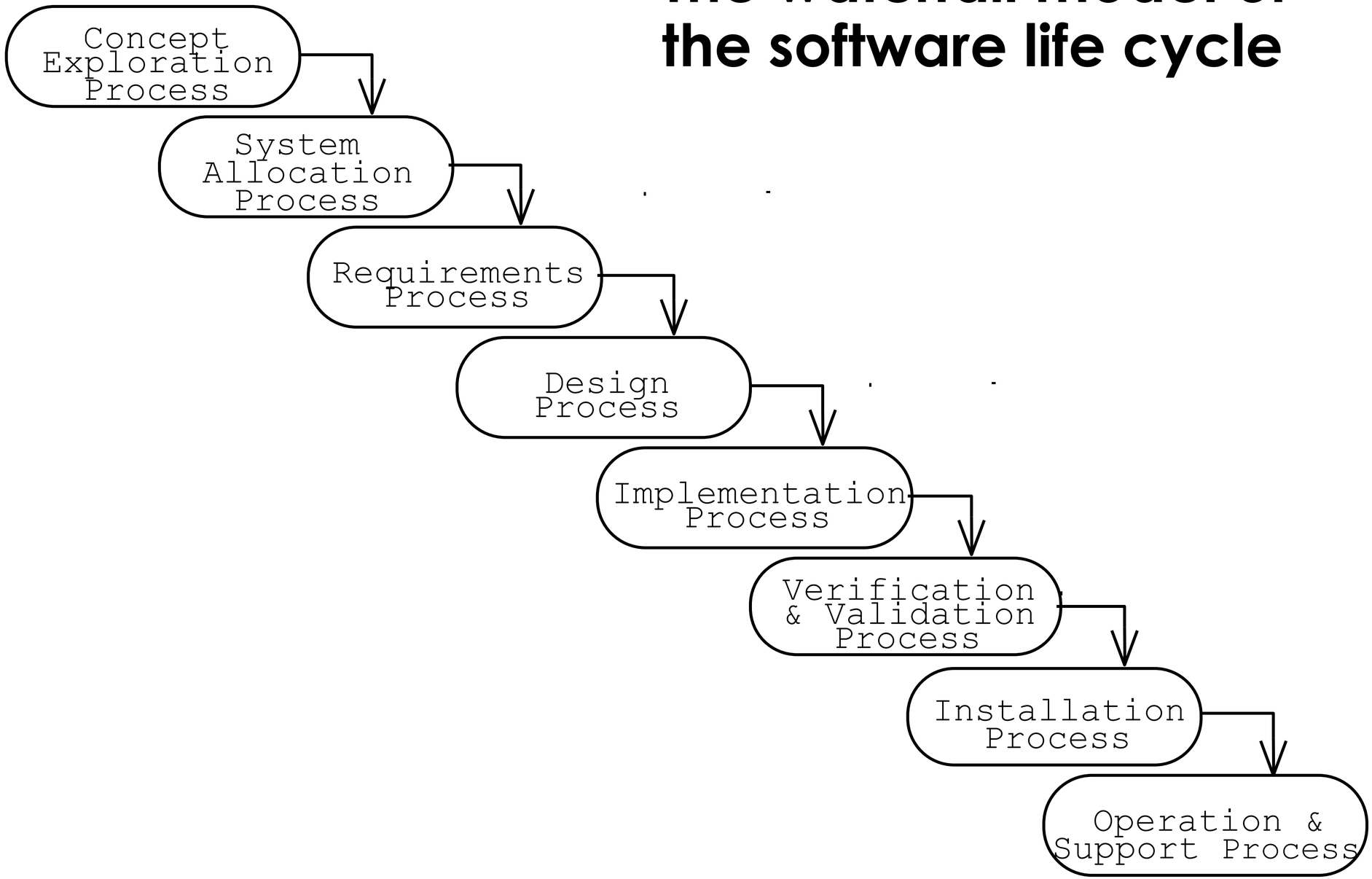
DevOps and Dependable Programmable Infrastructure

Software Lifecycle

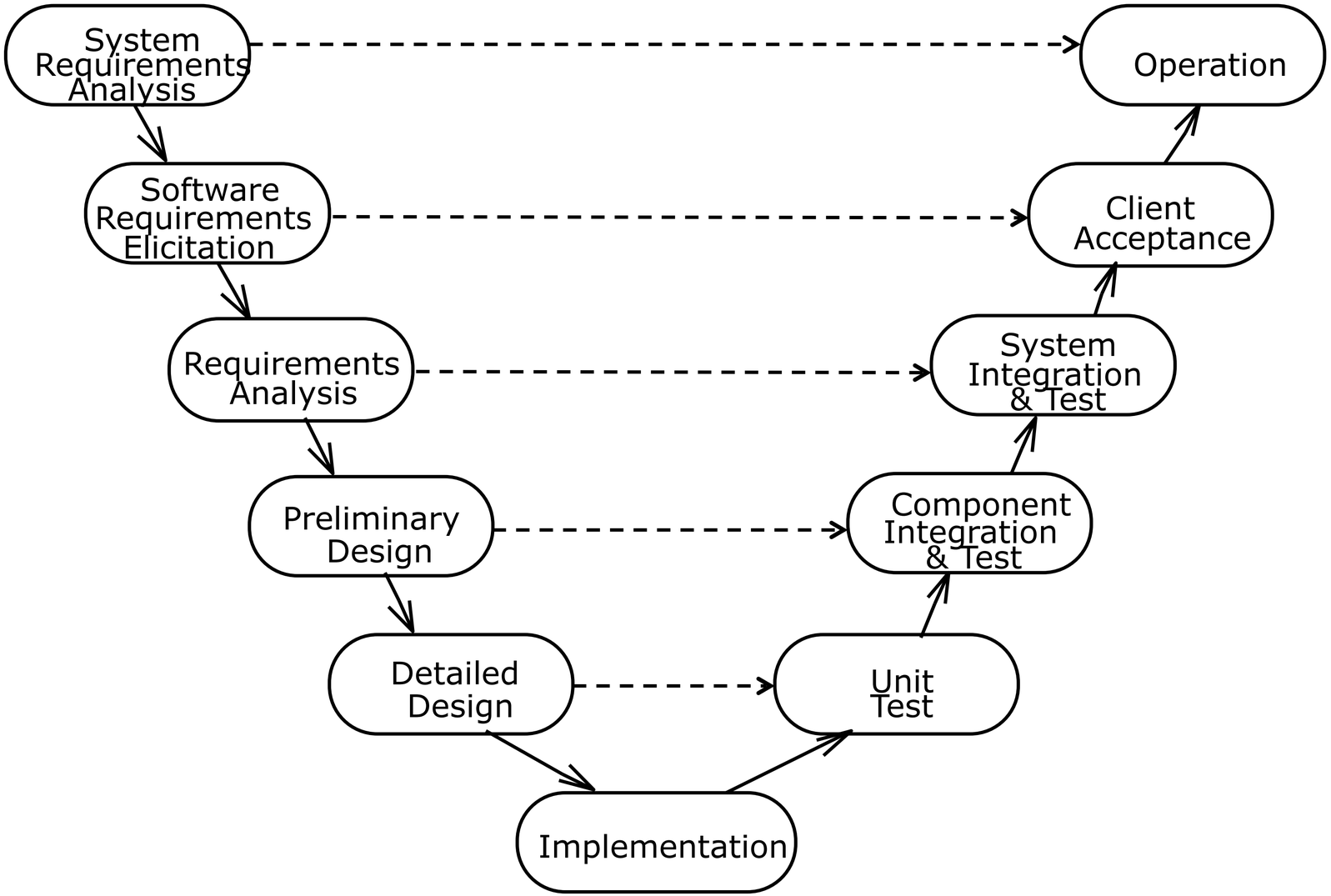
The term "life cycle" is based on the metaphor of a person's life.



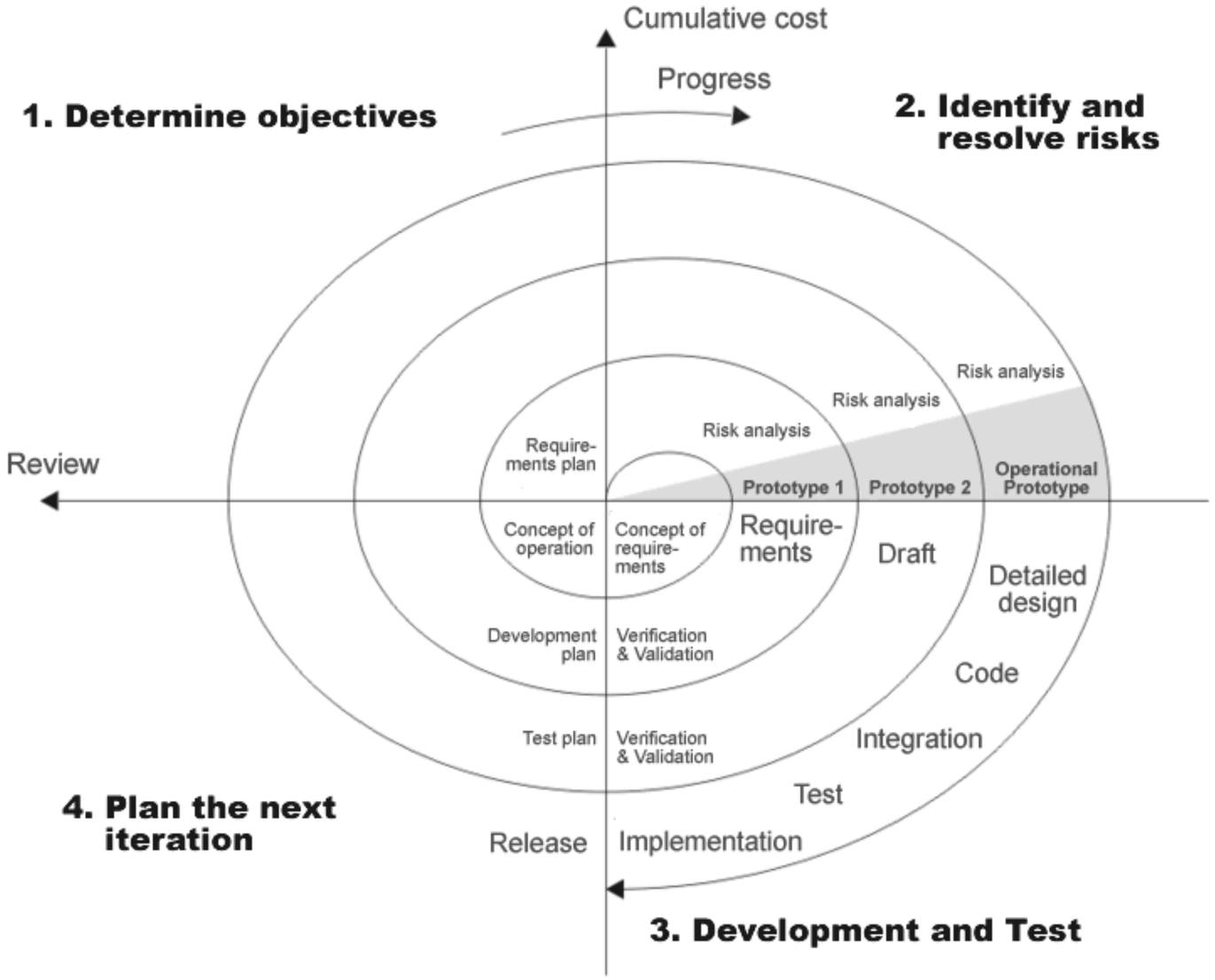
The waterfall model of the software life cycle



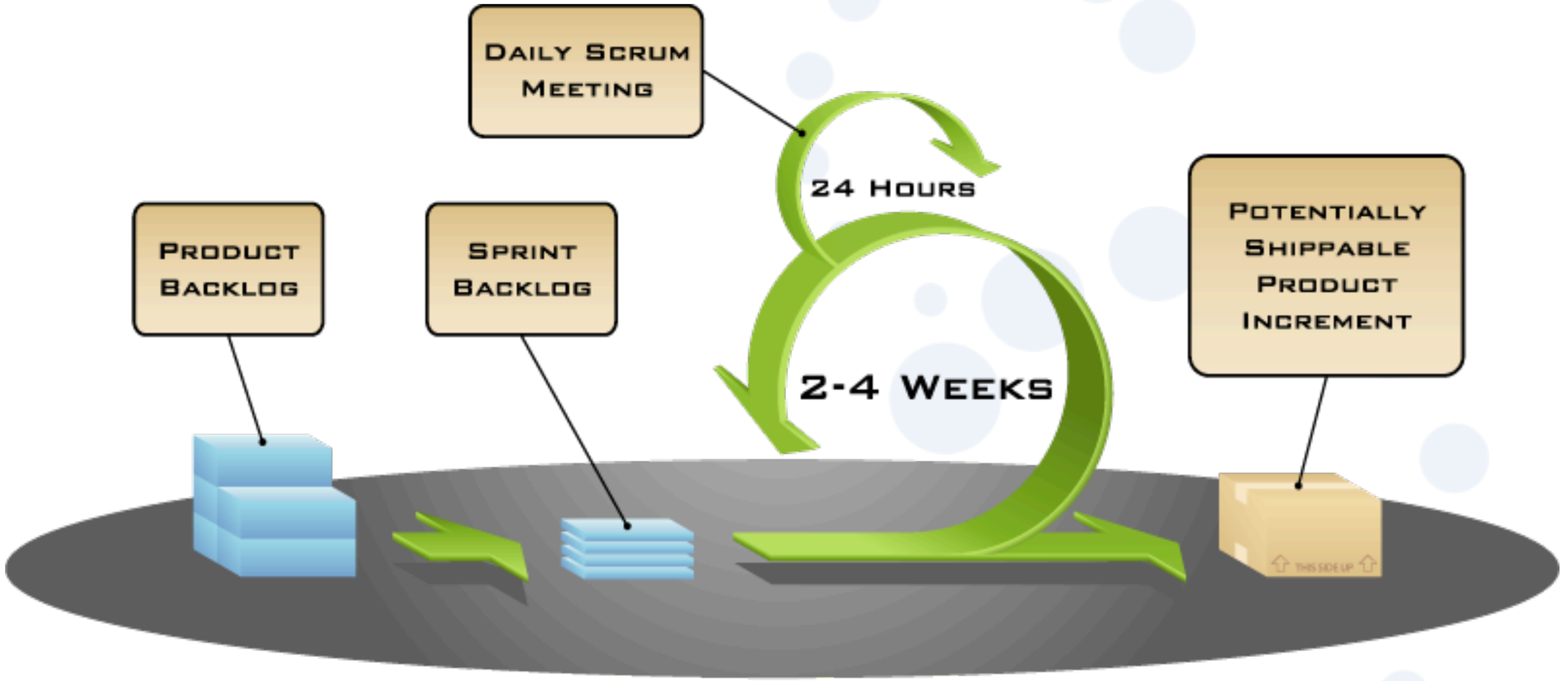
V-model activity diagram



Boehm's spiral model



Scrum


Daily Scrum Meeting
24 Hours
Product Backlog
Sprint Backlog
2-4 Weeks
Potentially Shippable Product Increment
Copyright © 2005, Mountain Goat Software

Which model to use?

PT = Project Time
MTBC = Mean Time Between Changes

Change is rare (MTBC » PT)
- Linear Model (Waterfall, V Model)
- Open problems are solved before starting a new phase.

Change occurs sometimes (MTBC ≈ PT)
- Iterative Model (Spiral Model)
- The change during a phase can lead to iterating a previous phase or canceling the project.

Change is frequent (MTBC « PT)
- Agile methods (e.g., Scrum)
- The phases never end, but happen in parallel.

Software development is only part of the equation!
Software must be put into production to meet business demands!

Back to 2013...



WALKER'S WORLD JULY 22, 2013 / 12:07 AM

Walker's World: Snov

By MARTIN WALKER, UPI Editor Emeri



China's version of Newsweek magazine featur
Snowden is sold at a news stand in Beijing on J

WASHINGTON, July 22 (UPI) --

If Edward Snowden had surfaced first in Brussels rather than in Hong Kong, maybe everything would have been different. He could have applied for asylum at the European Parliament.



Argentina's Jorge Mario Bergoglio elected Pope Francis I

Argentine Cardinal Jorge Mario Bergoglio has greeted crowds in Rome's St Peter's Square after his election as the Catholic Church's new Pope.

Appearing on a balcony over the square, he asked the faithful to pray for him. Cheers erupted as he gave a blessing.

The first Latin American and the first Jesuit to be pontiff, he will call himself Francis I.

Stuart Flinders reports.

13 March 2013 BBC News | Europe



Main software failure causes in 2013...

- Unrealistic deadlines (e.g., set by someone outside the development team)
- (Too) rapid changes in requirements and personnel
- Necessary resources estimated too optimistically
- Risks (technical and social) not taken into account at the start of the project
- Communication problems between staff members
- Management difficulties in recognizing recurring delays and making decisions
- Non-cooperative stakeholders



Gartner Report 2013

Main software failure causes in 2013...

HOW-TO

6 Software Development Lessons From Healthcare.gov's Failed Launch

The problems that plagued the launch of Healthcare.gov -- the online data hub and insurance marketplace central to healthcare reform -- will someday fill a book. For now, developers can apply these six lessons from Healthcare.gov's very public failure to their own software projects.

By Matt Heusser

CIO | NOV 18, 2013 7:00 AM PST

> "I spent $174 million on a website and all I got was this bad press."
>
> — Someone, somewhere in the U.S. Department of Health and Human Services (HHS)

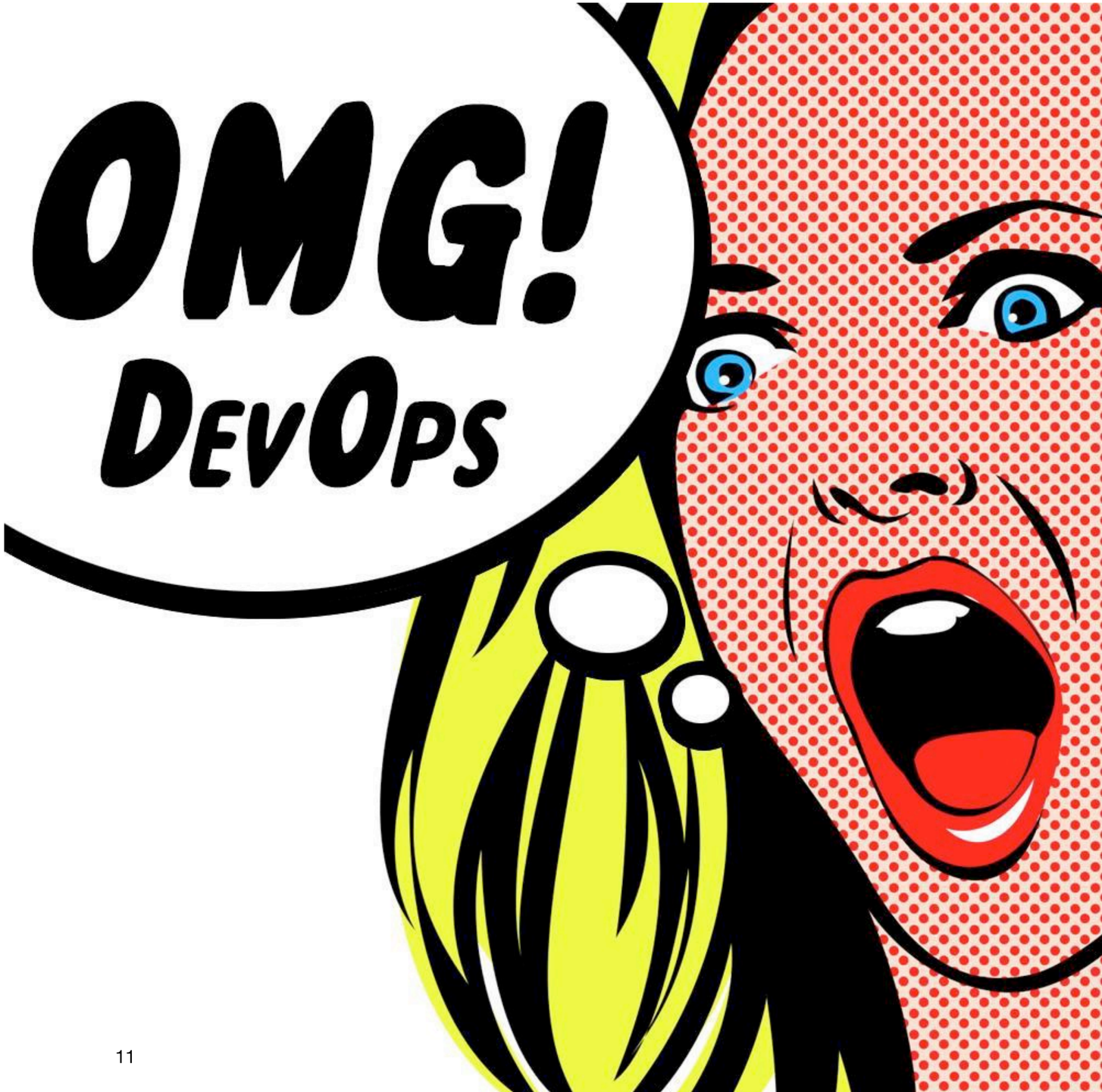
OBAMACARE



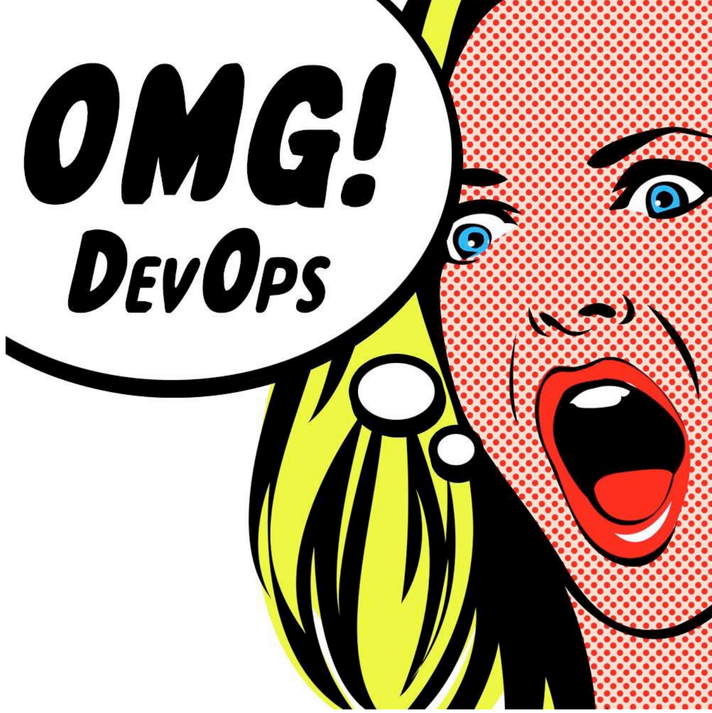
http://www.cio.com/article/2380827/developer/6-software-development-lessons-from-healthcare-gov-s-failed-launch.html

"DevOps is a set of practices aimed at reducing the time between committing a change to a system and the change in normal production, while ensuring high quality."

L. Bass et al.



DevOps



In an organization not using DevOps, there are two teams.

The Dev team takes care of

- Requirements development and testing
- Requirements changes
- Release of products

The Ops team takes care of

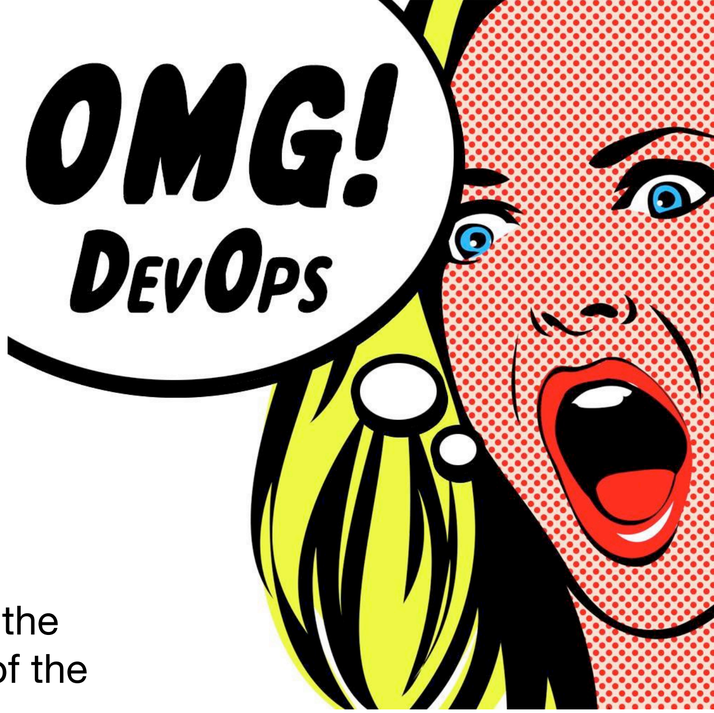
- Offer services to customers
- Provide guarantees and stability to the service

"The system works correctly in the development machine but not in the operating machine"

"10 days after the correct distribution of the latest version, the system is overloaded"

Who is the culprit? The Dev team or the Ops team?

DevOps



The main problem is that the two groups are totally disconnected!

- The Dev team aims to provide new versions of the software
- The Dev team pays no attention to Quality of Service

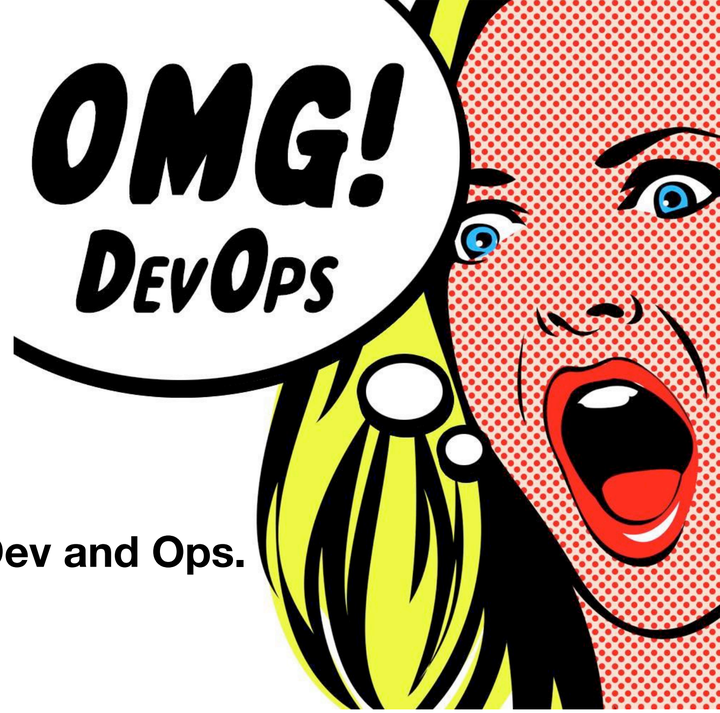
- The Ops team aims to delay the production of new versions of the software
- The Ops team aims to guarantee the Quality of Service

Software changes are critical for business!

Quality of Service is also essential!

DevOps provides practices and tools that bridge the gap between Dev and Ops.

DevOps



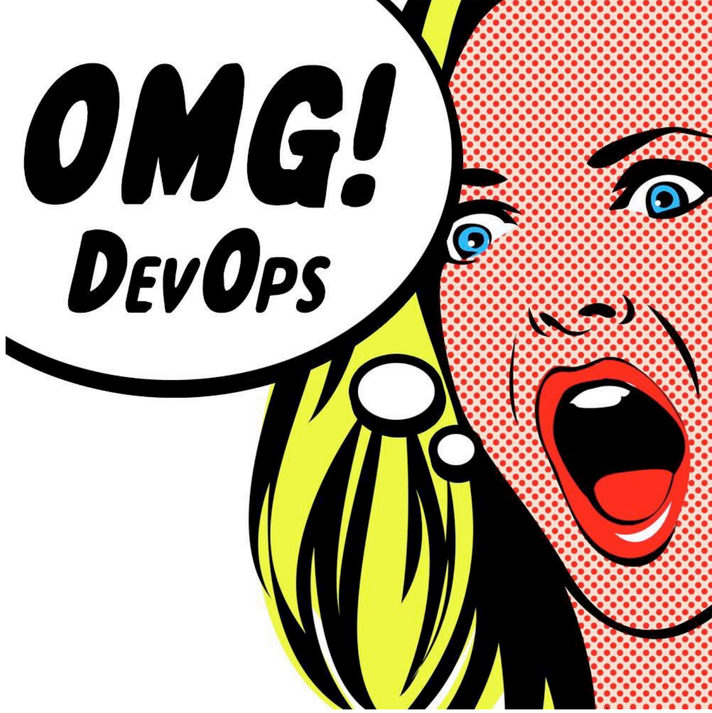
DevOps provides practices and tools that bridge the gap between Dev and Ops.

Objective: To create a collaborative mindset in which a single team dealing with both Dev and Ops.
The team must have different skills, different backgrounds, etc.

DevOps requires:

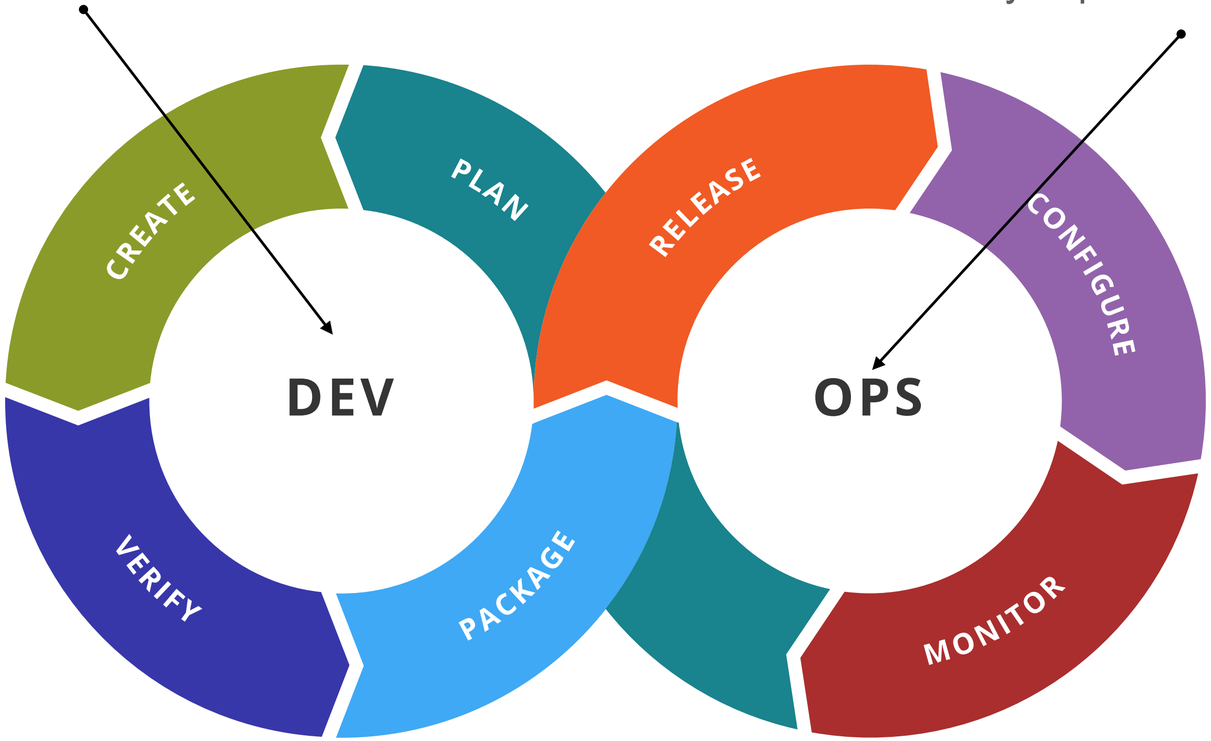
- Culture management;
- Automation tools;
- Both organizational and technical metrics;
- Continuous sharing of artifacts, procedures, languages, approaches ...

DevOps

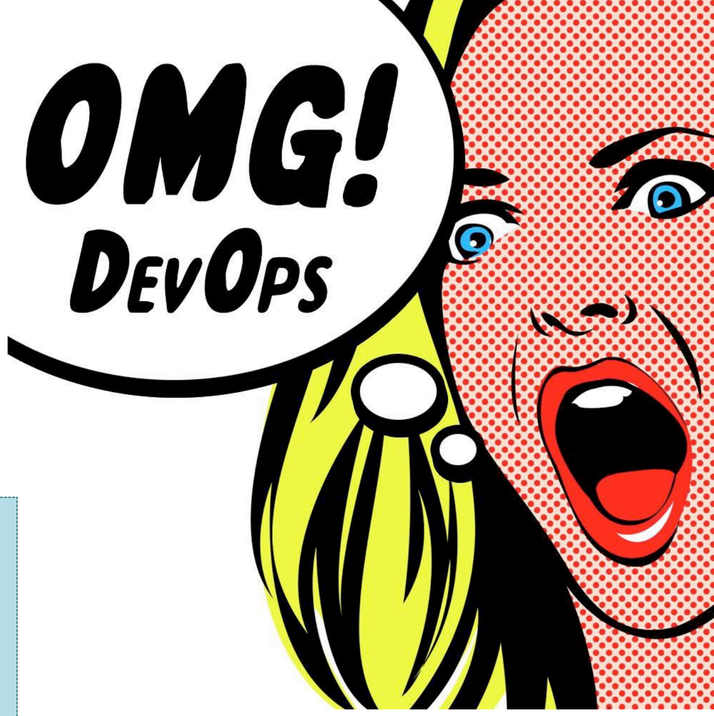


Dev's goal:
"Prepare a software architecture designed to be immediately implementable"

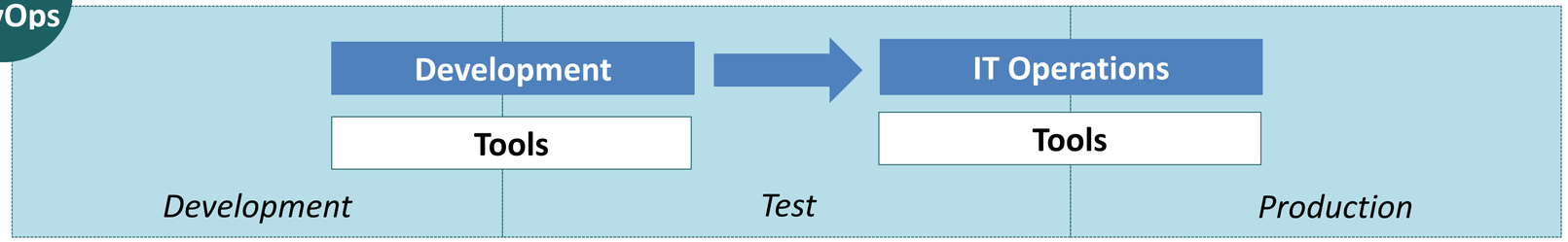
Ops's goal:
"Observe the runtime of the architecture and provide operational feedback to the developer to continuously improve the architecture"



DevOps

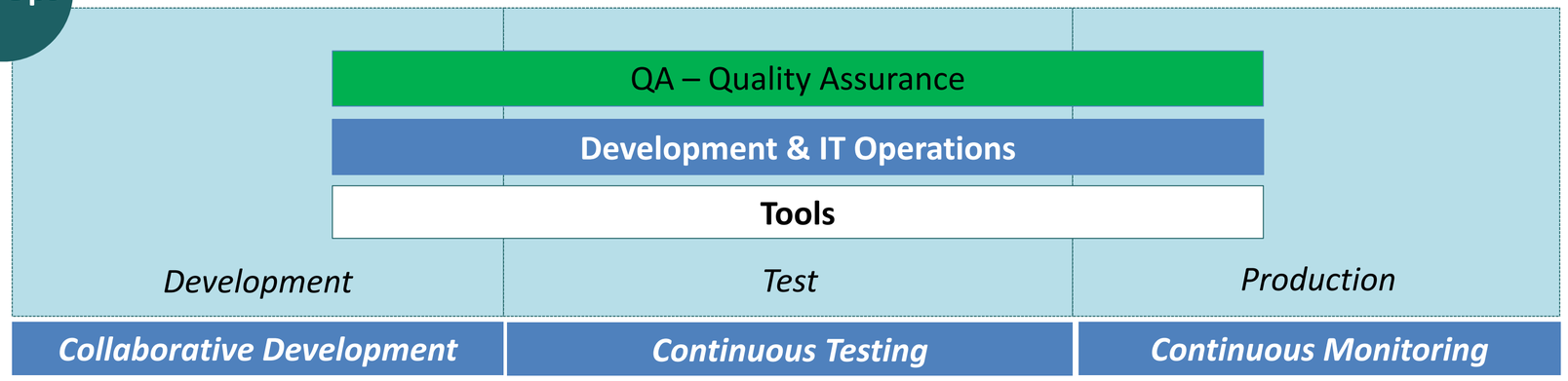


Before DevOps

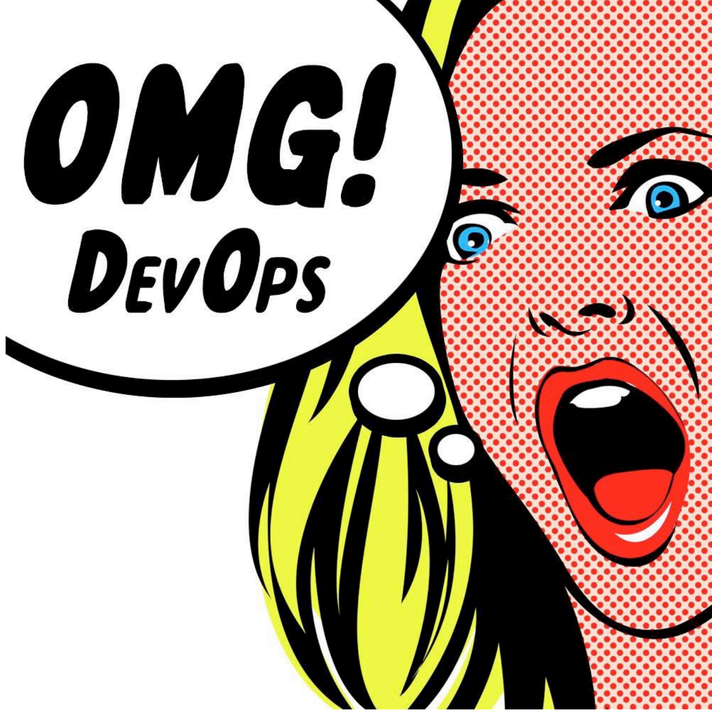


«*Devs* have to walk in *Ops* shoes and viceversa»

DevOps



DevOps



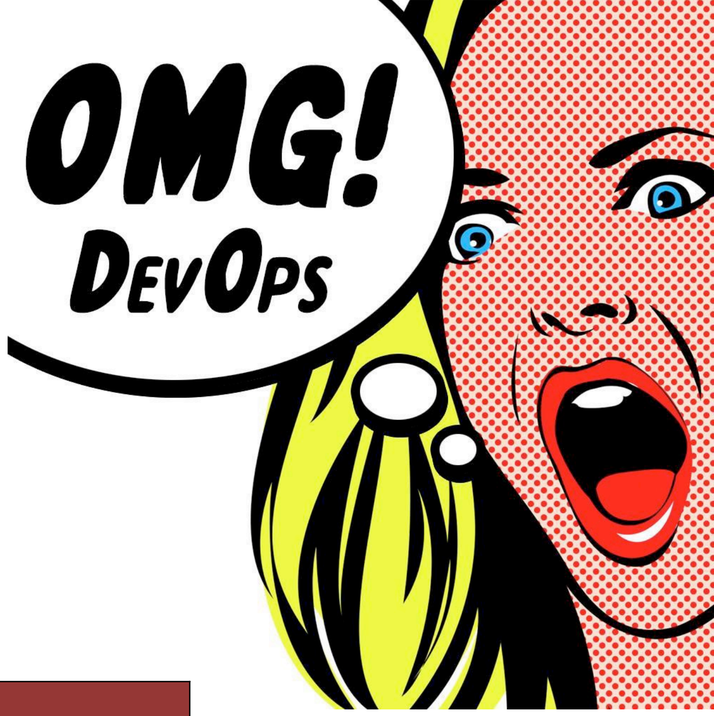
Development and testing in a production-like environment

Accelerated deployment using suitable processes and tools

Continuous collaboration and feedback

Continuous QoS validation

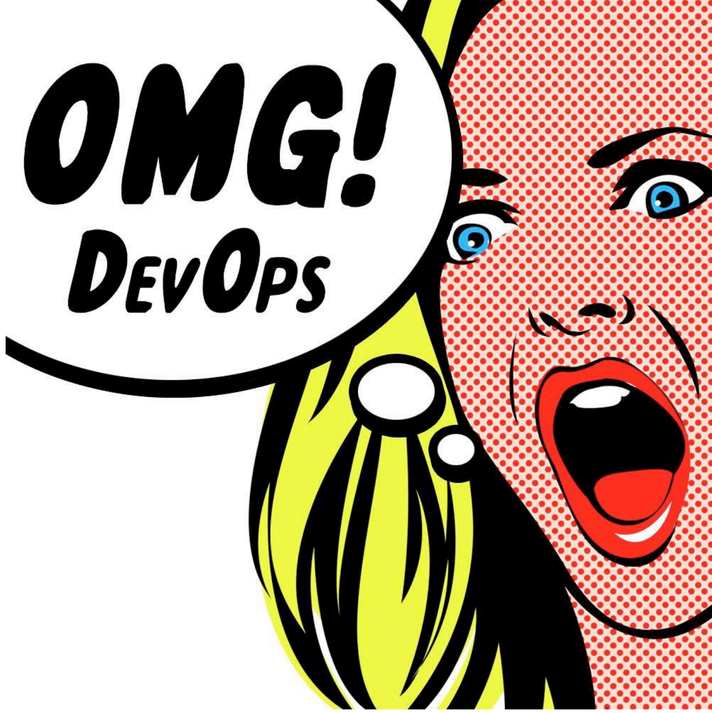
DevOps



DevOps practice examples that improve IT performance

Throughput	Stability & Quality
Distribution frequency • Continuous delivery • Version control	Average recovery time • Version control • Application and system monitoring
Delivery time for new versions • Version control • Continuous architecting • Continuous integration • Continuous testing	

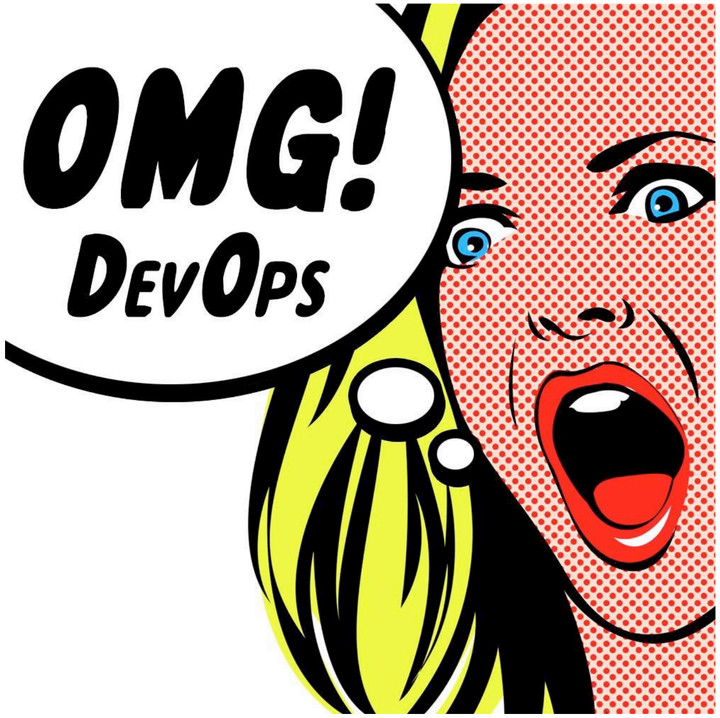
DevOps



Waterfall Model vs. Agile vs. DevOps

Problem	Waterfall Model	Agile	DevOps
Main Focus	Design and Development	Design and Development	**Full Application Lifecycle**
Focus on Ops	None or minimal	None or minimal	**Main focus**
Maintenance Role	It is considered another development phase	It is considered another development phase	**It replaces a system in production with another**
Process Result	A packaged system ready to be released	*Various intermediate demo applications*	**A continuously updated system in production**
Quality Assurance	Testing usually takes place at the end of the process	*Test-driven development*	**Test-driven development and monitoring-driven operations that give feedback to the Dev team**

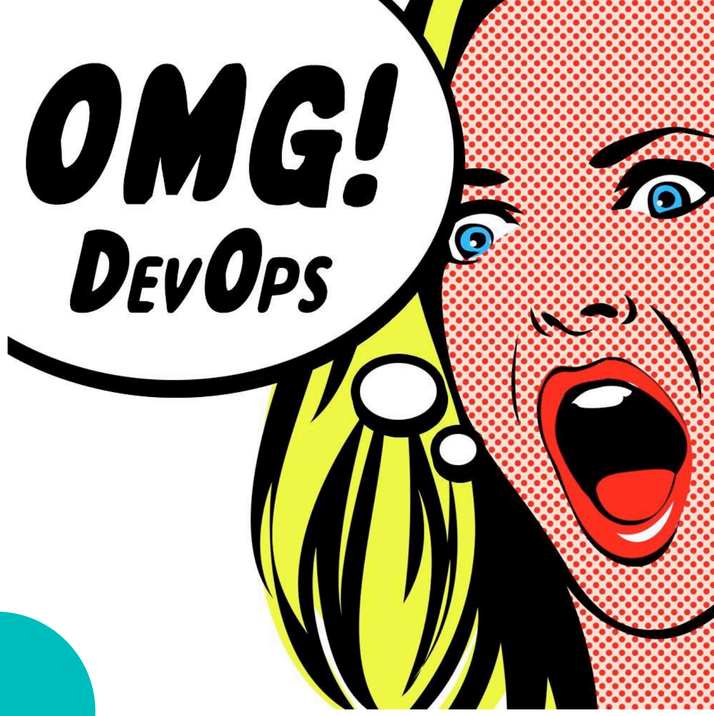
DevOps



The advantages of adopting DevOps

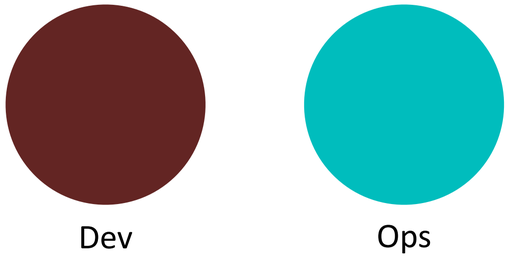
- Shorter time to value;
- Fast and regular new software releases;
- Better quality;
- Risks mitigated;
- Greater collaboration in the team.

DevOps

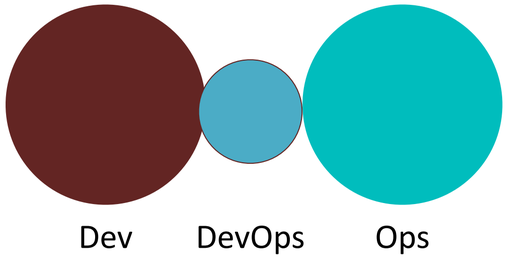


The disadvantages are related to organizational changes

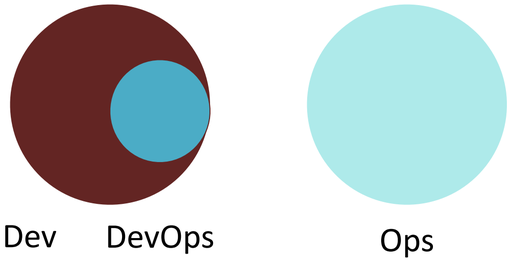
What to avoid



Separate Silos



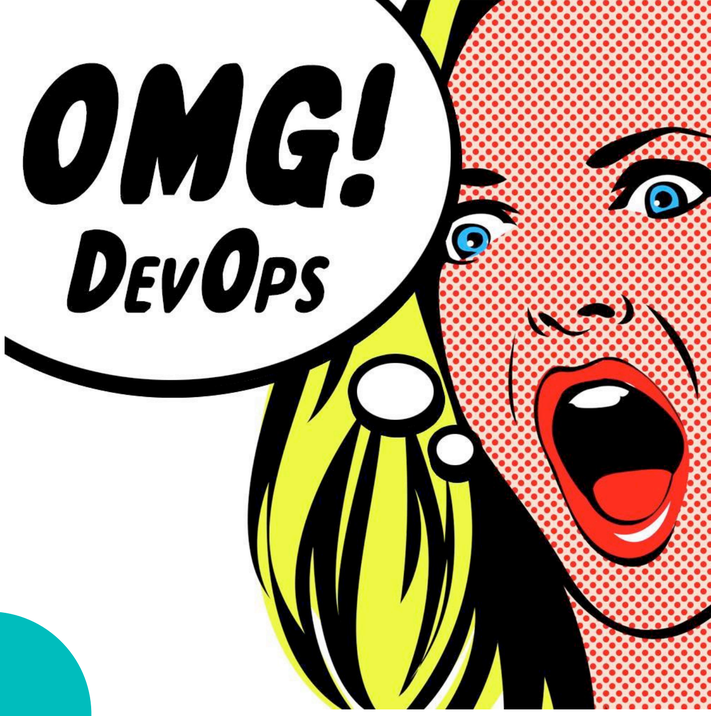
Separate DevOps Silo



We don’t need Ops

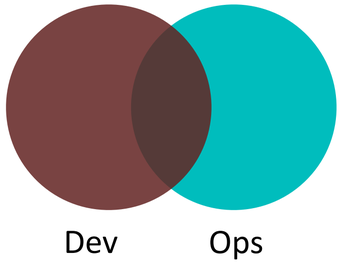
From M. Skelton. What Team Structure is Right for DevOps to Flourish? 2013
http://blog.matthewskelton.net/2013/10/22/what-team-structure-is-right-for-devops-to-flourish/
http://web.devopstopologies.com

DevOps



The disadvantages are related to organizational changes

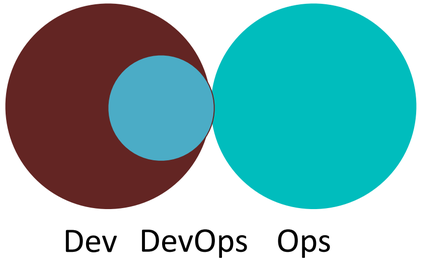
What to promote



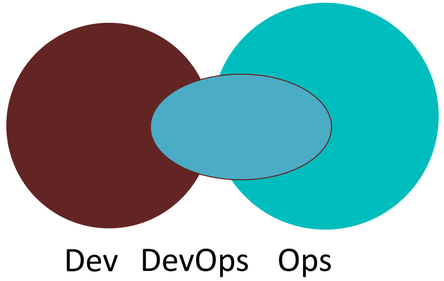
Smooth collaboration



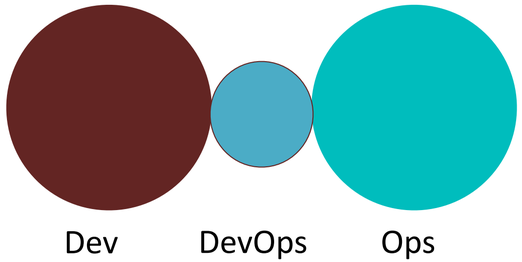
Fully Embedded



Infrastructure as a Service



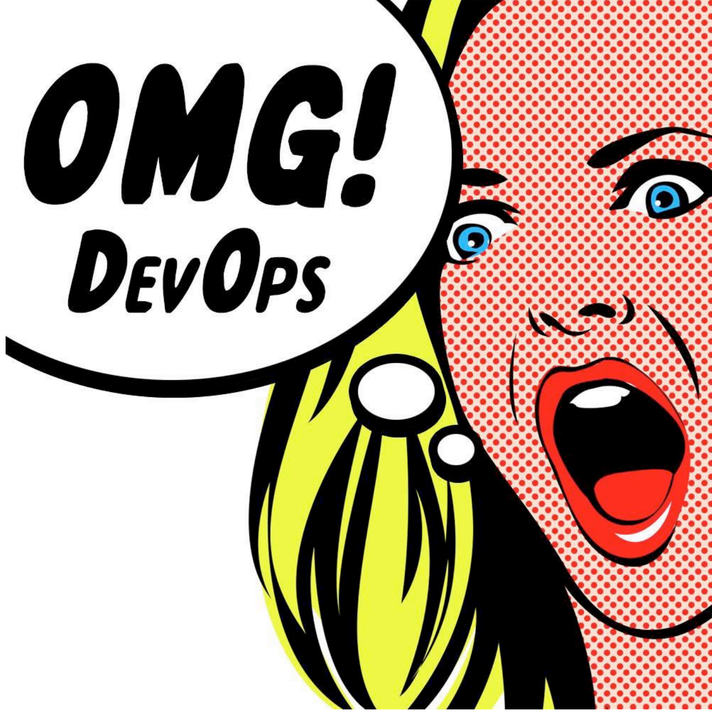
DevOps as a Service

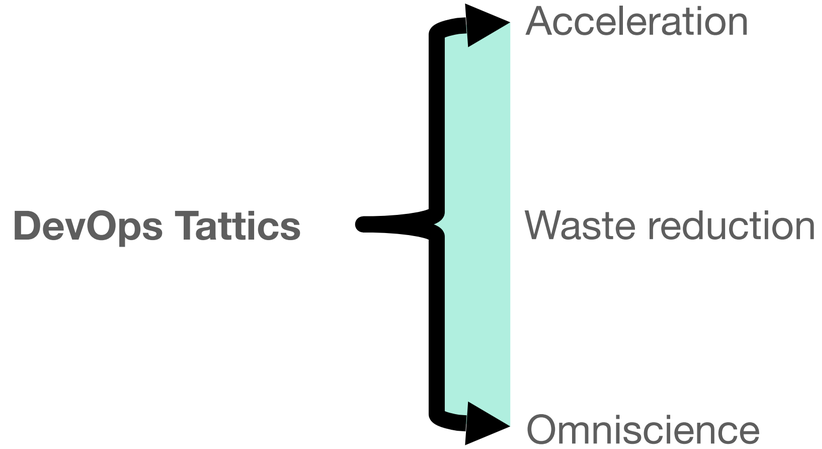


Temporary DevOps Team

From M. Skelton. What Team Structure is Right for DevOps to Flourish? 2013
http://blog.matthewskelton.net/2013/10/22/what-team-structure-is-right-for-devops-to-flourish/
http://web.devopstopologies.com

DevOps tattics





Tactics for "Acceleration"

- Merging the Dev and Ops Teams
- Infrastructure-as-Code
- Use of Continuous Testing tools
- Use of Continuous Integration tools
- Use of Continuous Deployment tools
- ...

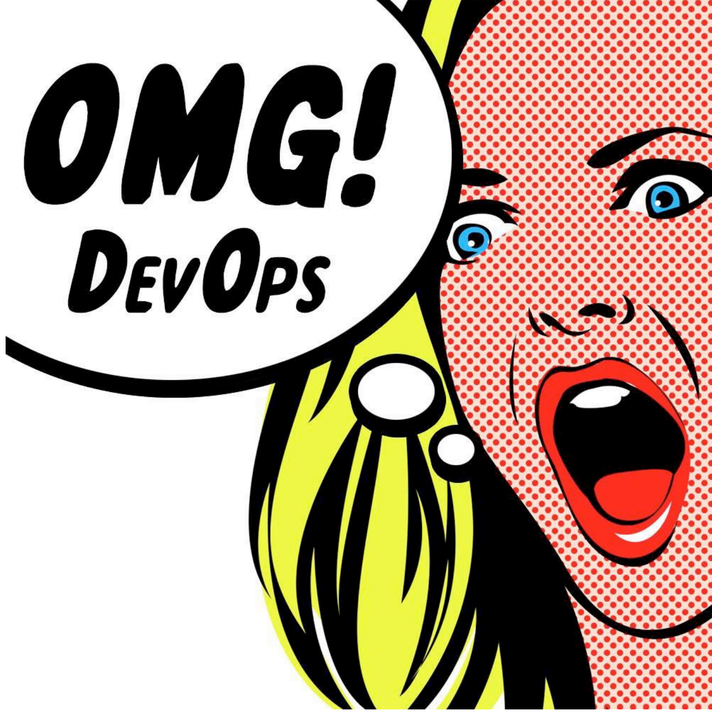
Tactics for "Waste Reduction"

- Canary Testing
- A / B Testing
- Reduction of documentation
- Minimal architectures ➔ Microservices
- ...

Tactics for "Omniscience"

- Monitoring of all socio-technical aspects also called Omniscient DevOps Analytics
- Monitoring-as-a-service
- Continuous risk analysis and prevention
- Continuous improvement of the architecture
- ...

Architectures & DevOps



All software architectures meet specific requirements.

A software architecture for DevOps must have the following requirements:

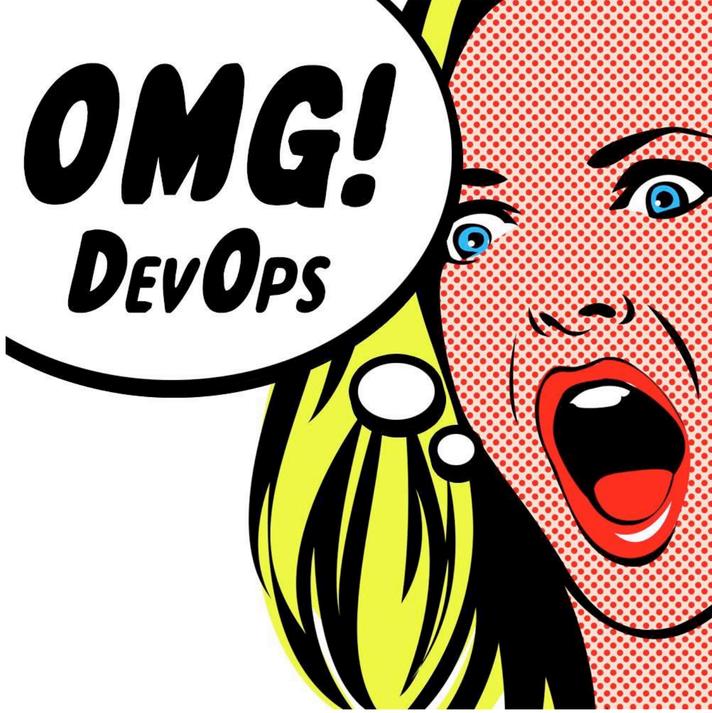
- Modifiability;
- Observability;
- Rapid evolution;
- Testability;
- High scalability;

but most of all

- Failure of the current software architecture!

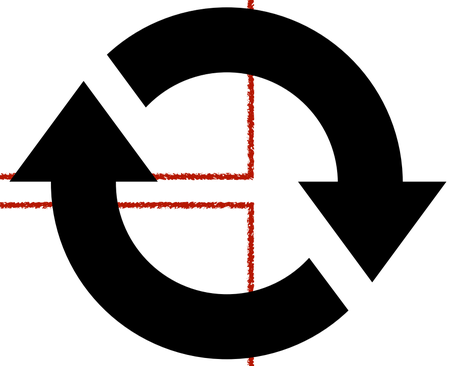
The software architectures in DevOps are subject to continuous improvement which must take into account maintainability!

The maintenance challenge



The maintenance cost constitutes ~70% of the total cost of a software system.

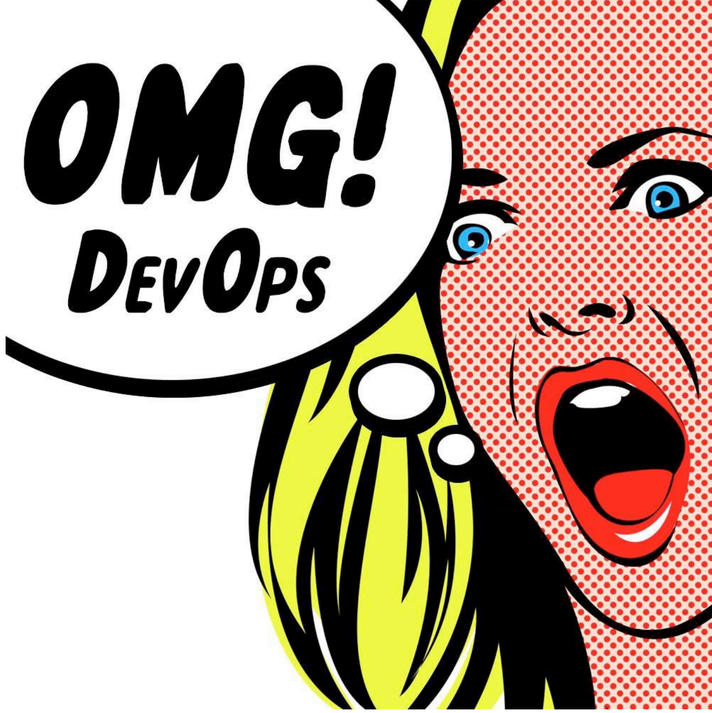
The number of small systems continuously distributed is growing
- Systems need to be maintained
- The pipelines that join the systems must be maintained
- The maintenance of the two previous points are concurrent



Socio-technical alignment

Larger and more diverse organizations
- "Maintenance" of the team
- Measurement and analysis of organizational structures

Processes and practices



Continuous Architecting

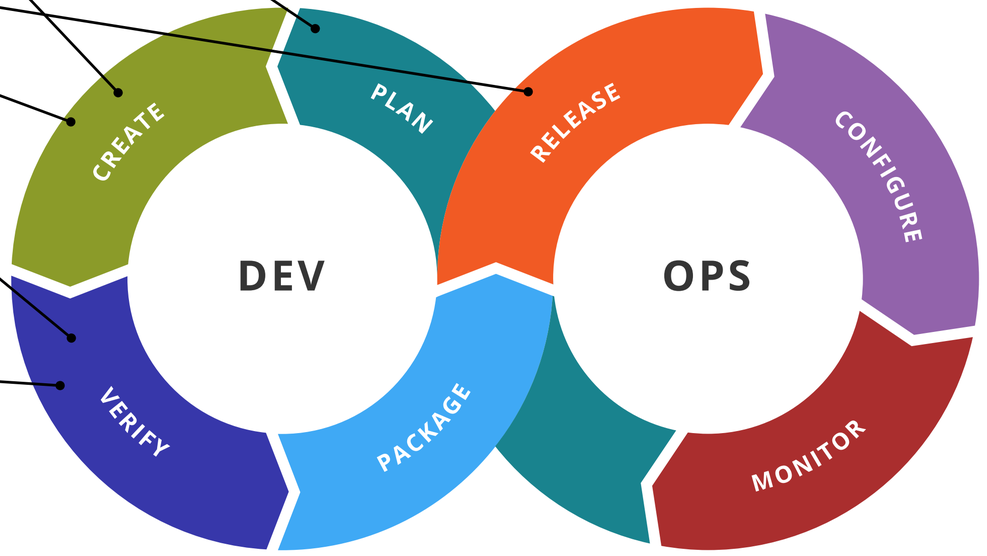
Def. *"Architect to test, build and deploy, taking quality attributes into consideration and leveraging runtime feedback"*

Continuous Integration

Def. *"Frequently merge the working copies of all developers into a shared main line"*
e.g., Apache Jenkins, Travis-CI.

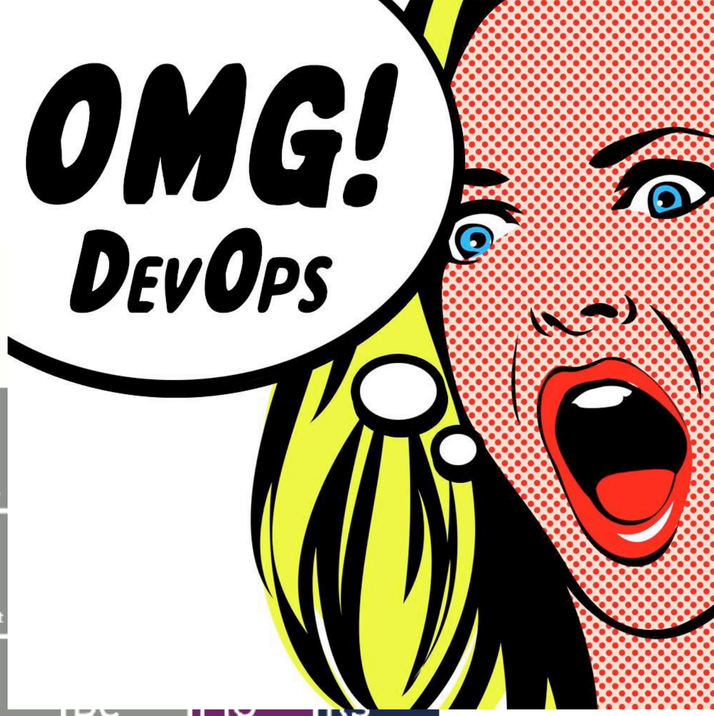
Continuous Testing

Def. *"Run tests as part of the build pipeline so that each distribution is validated"*
e.g., Selenium + JUnit.



Tools

https://digital.ai/learn/devops-periodic-table/

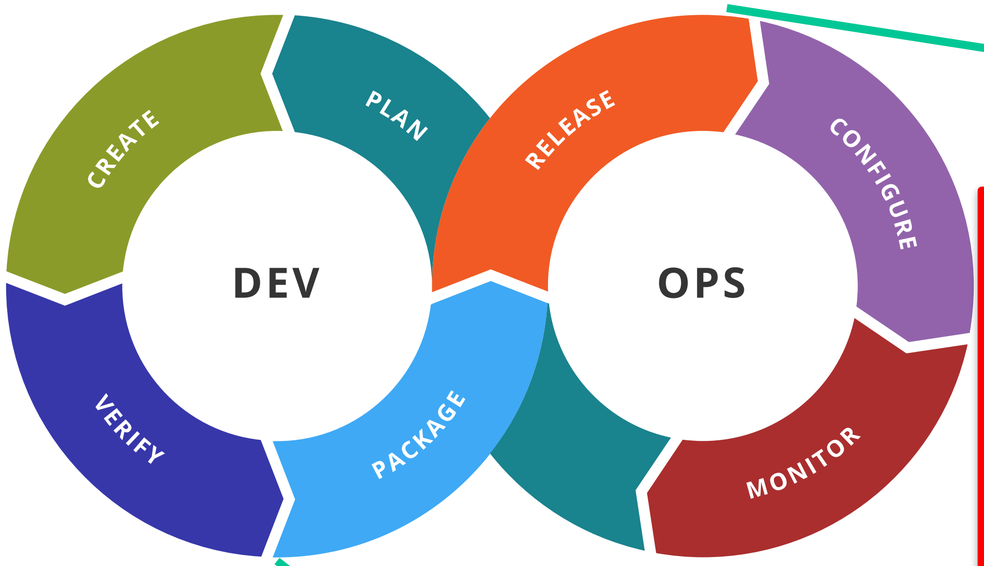


PERIODIC TABLE OF DEVOPS TOOLS (V2)

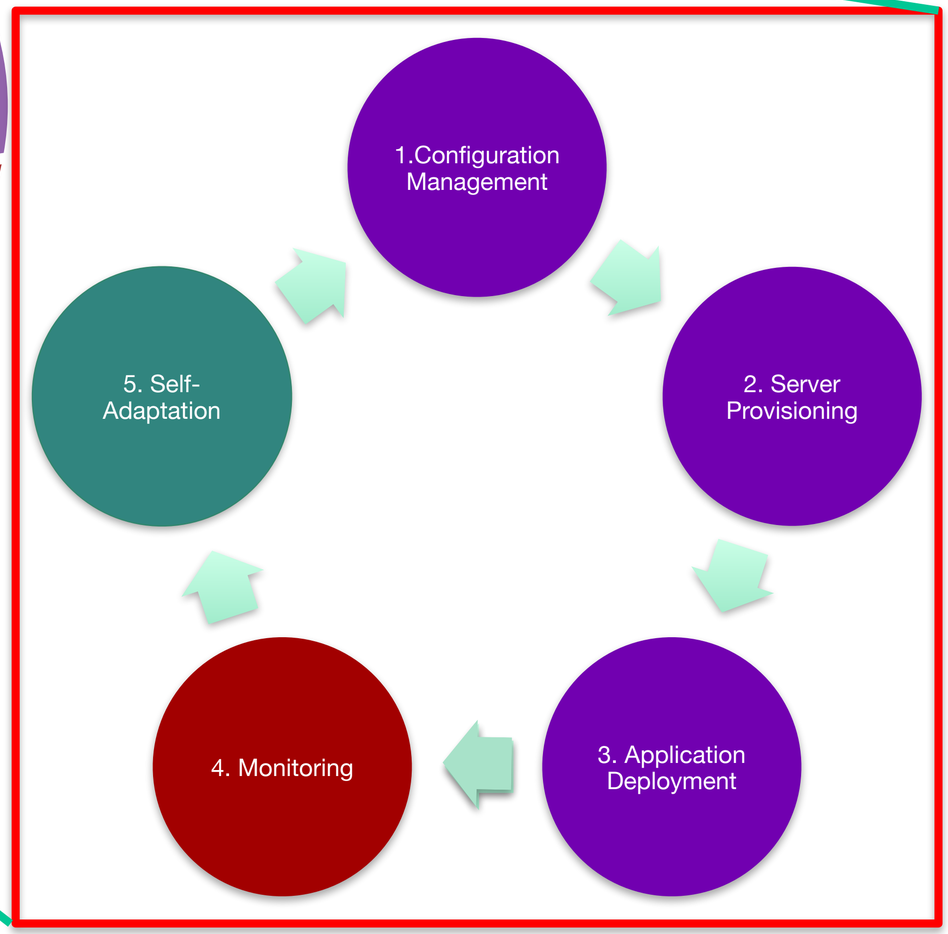
Os Open Source	SCM	Database Mgmt	Build
Fr Free	CI	Repo Mgmt	Testing
Fm Freemium	Deployment	Config / Provisioning	Containerization
Pd Paid	Cloud / IaaS / PaaS	Release Mgmt	Collaboration
En Enterprise	BI / Monitoring	Logging	Security

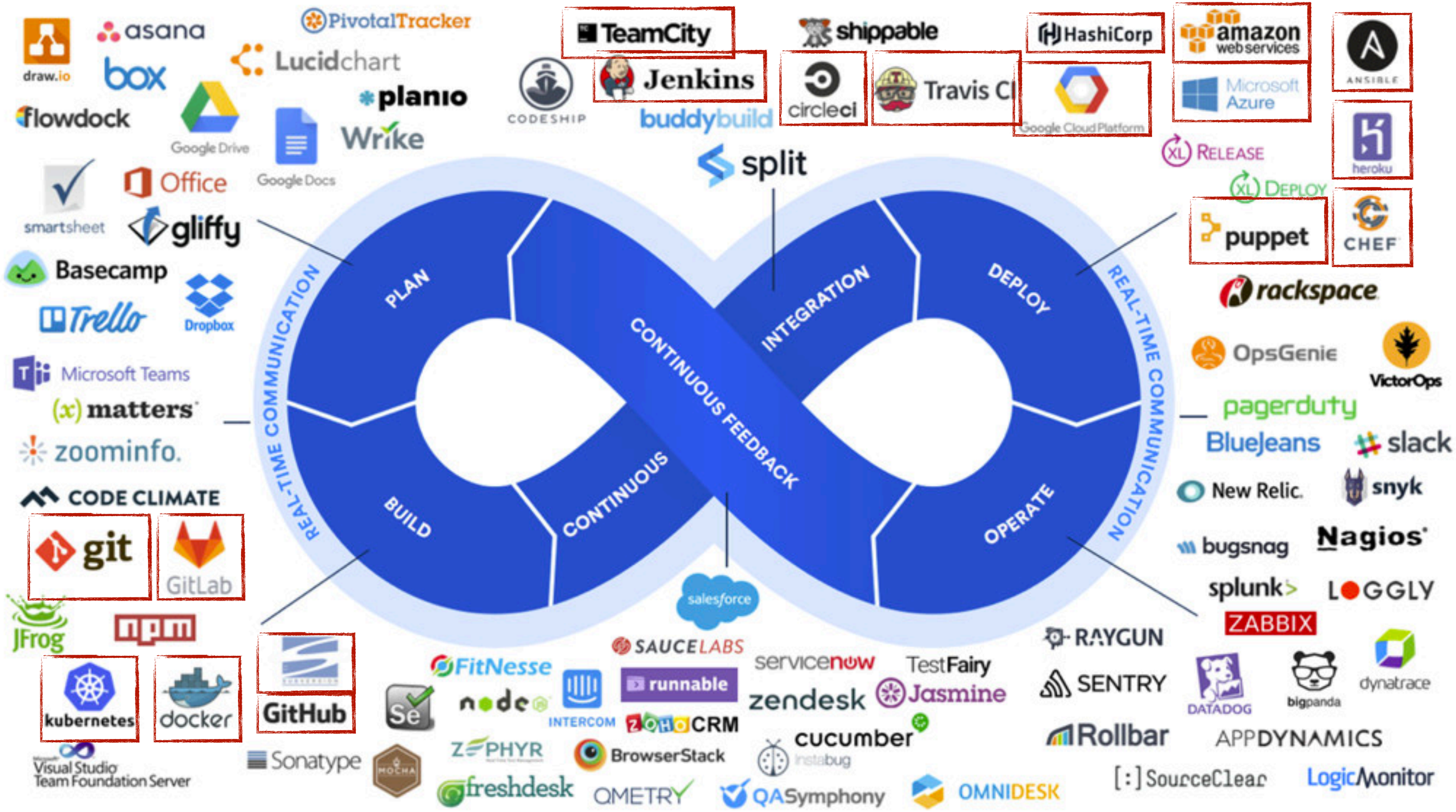
| | | |

The DevOps toolchain

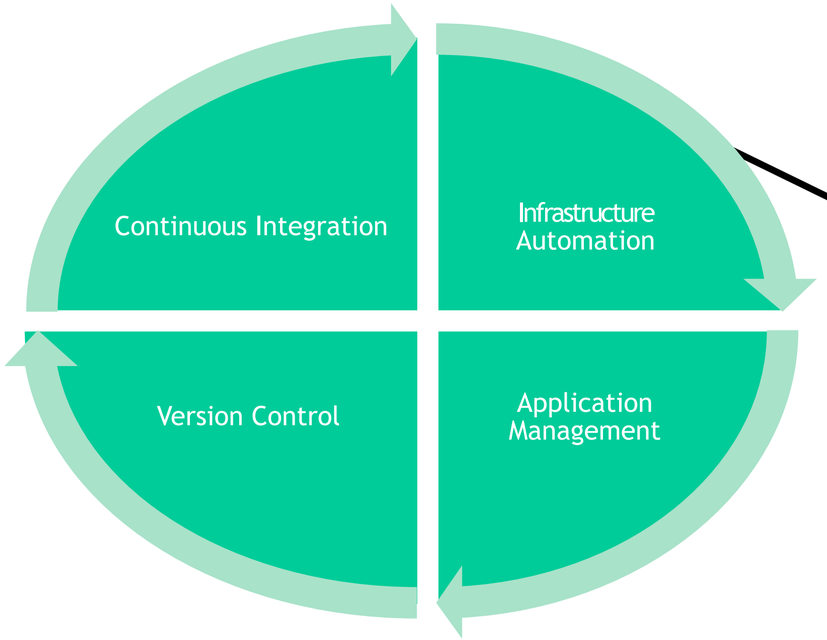


Continuous Architecting

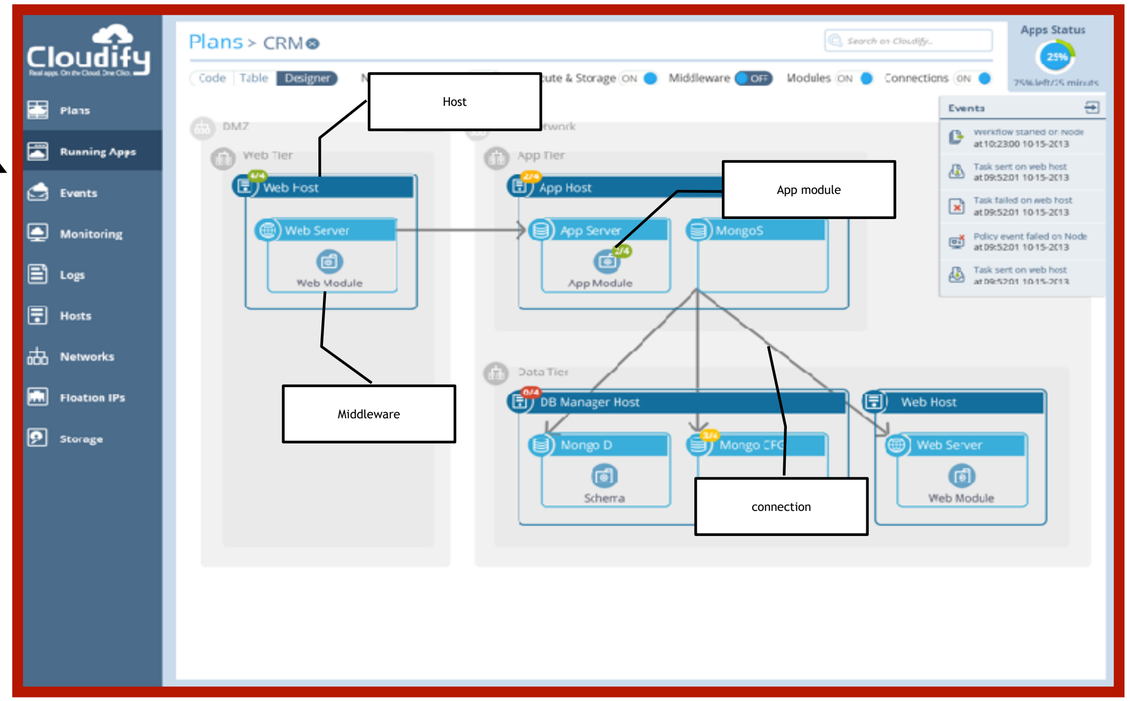



PLAN
CONTINUOUS
INTEGRATION
DEPLOY
BUILD
CONTINUOUS
OPERATE
CONTINUOUS FEEDBACK
REAL-TIME COMMUNICATION
REAL-TIME COMMUNICATION
draw.io
asana
box
PivotalTracker
Lucidchart
planio
flowdock
Google Drive
Google Docs
Wrike
smartsheet
Office
gliffy
Basecamp
Trello
Dropbox
Microsoft Teams
(x) matters
zoominfo.
CODE CLIMATE
git
GitLab
JFrog
npm
kubernetes
docker
GitHub
Visual Studio Team Foundation Server
Sonatype
CODESHIP
TeamCity
Jenkins
buddybuild
shippable
circleci
Travis CI
split
HashiCorp
Google Cloud Platform
amazon web services
Microsoft Azure
ANSIBLE
heroku
RELEASE
DEPLOY
puppet
CHEF
rackspace
OpsGenie
VictorOps
pagerduty
BlueJeans
slack
New Relic.
snyk
bugsnag
Nagios
splunk>
LOGGLY
ZABBIX
RAYGUN
SENTRY
DATADOG
bigpanda
dynatrace
Rollbar
APPDYNAMICS
[:] SourceClear
LogicMonitor
salesforce
SAUCELABS
FitNesse
node
runnable
INTERCOM
ZOHO CRM
ZEPHYR
BrowserStack
freshdesk
QMETRY
servicenow
TestFairy
zendesk
Jasmine
cucumber
instabug
QASymphony
OMNIDESK

Infrastructure Automation



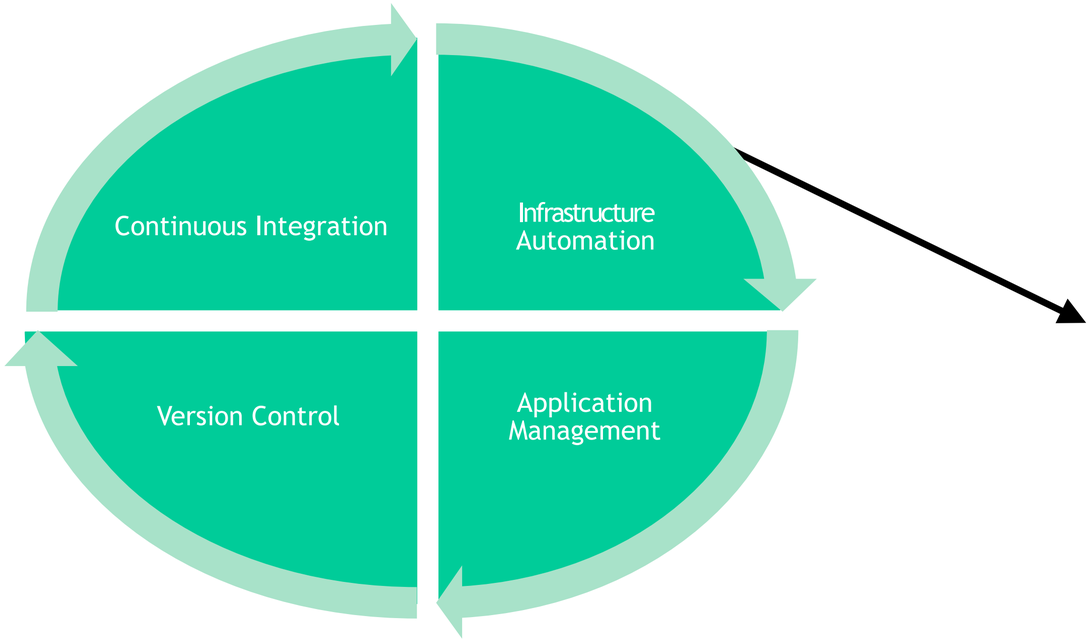
Infrastructure Automation ➔ a deployment script for any cloud application.

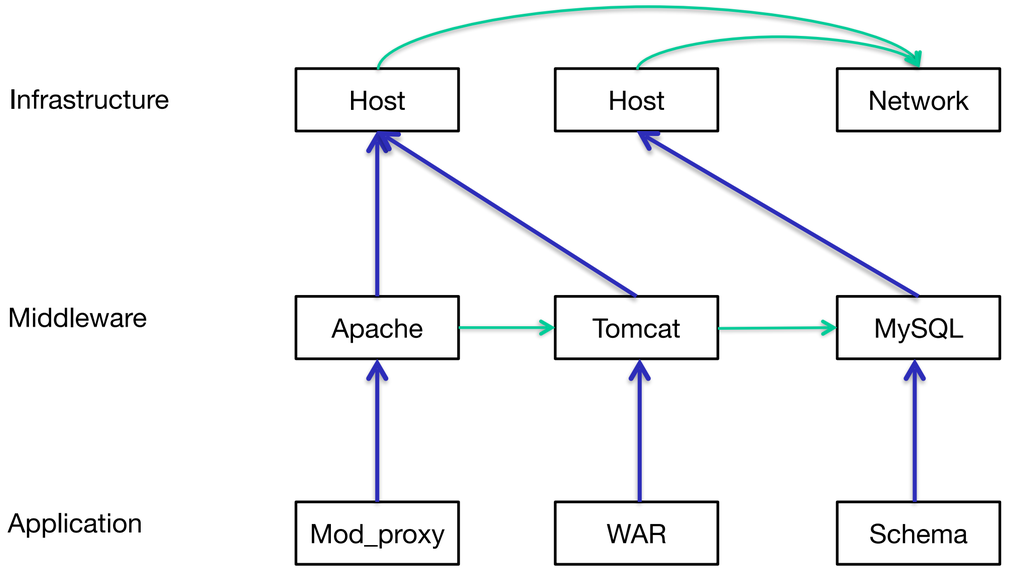


Note: this is a “simple” web application, e.g., a website!

What do we see here? I.e. what’s behind the curtain?

Infrastructure Automation





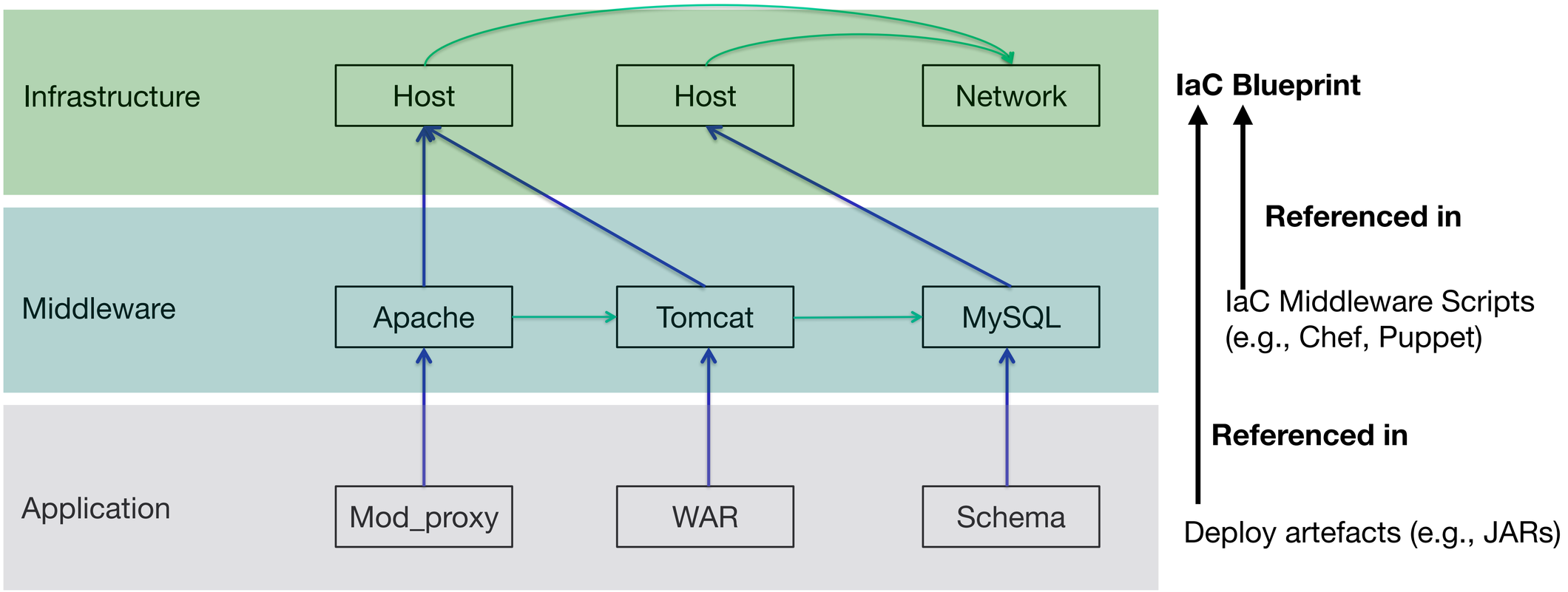
Infrastructure Automation ➔ a deployment script for any cloud application.

An Application Deployment Topology, i.e., “a graph of physical artifacts that need support for several lifecycle phases (e.g., procurement, installation, configuration, deployment, teardown)”

Toward Infrastructure-as-Code

An Application Deployment Topology, i.e., “a graph of physical artifacts that need support for several lifecycle phases (e.g., procurement, installation, configuration, deployment, teardown)”

Infrastructure-as-code, i.e., “a blueprint detailing physical artifacts, all scripts for all lifecycle phases, and all artifacts needed for deployment.”



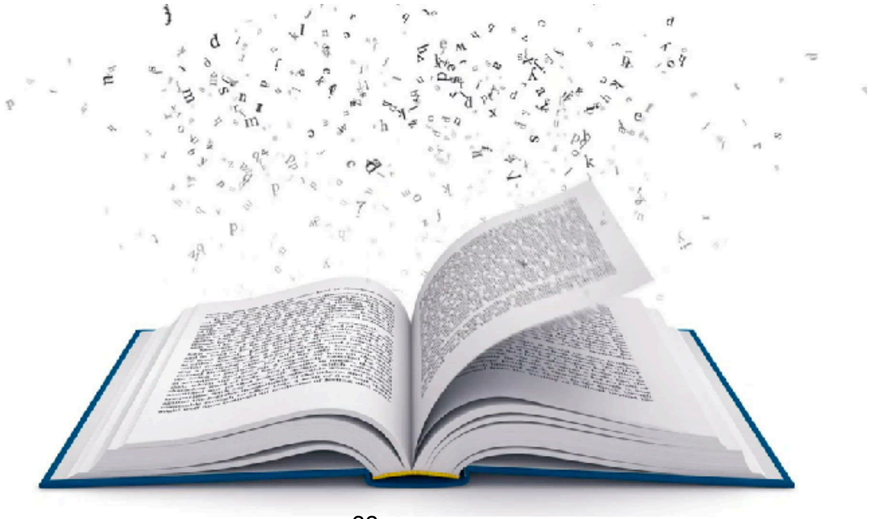
Infrastructure as Code

*“Infrastructure-as-Code (IaC) is a model for **provisioning and managing** a computing environment using the explicit definition of the desired state of the environment in source code and applying software engineering principles, methodologies, and tools.*

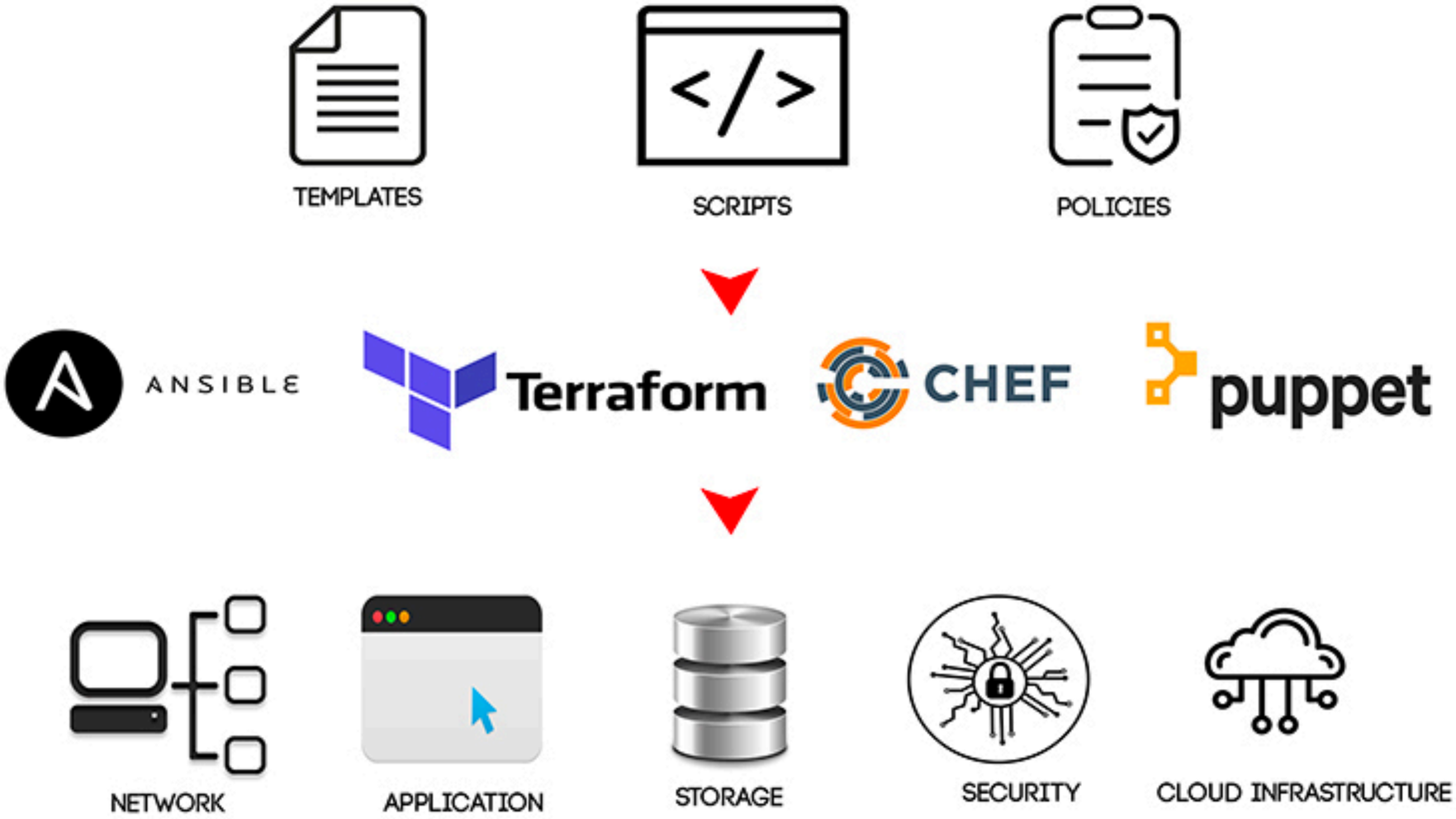
IaC DSLs enable defining the environment state as a software program, and IaC tools enable managing the environment based on such programs.

*The managed computing environment comprises **three key types of computing resources** (i.e., infrastructure, software/platform, and application) and exhibits **six essential properties** (i.e., Virtualized, Programmable, Consistent, Immutable, Auditable, and Reproducible).*

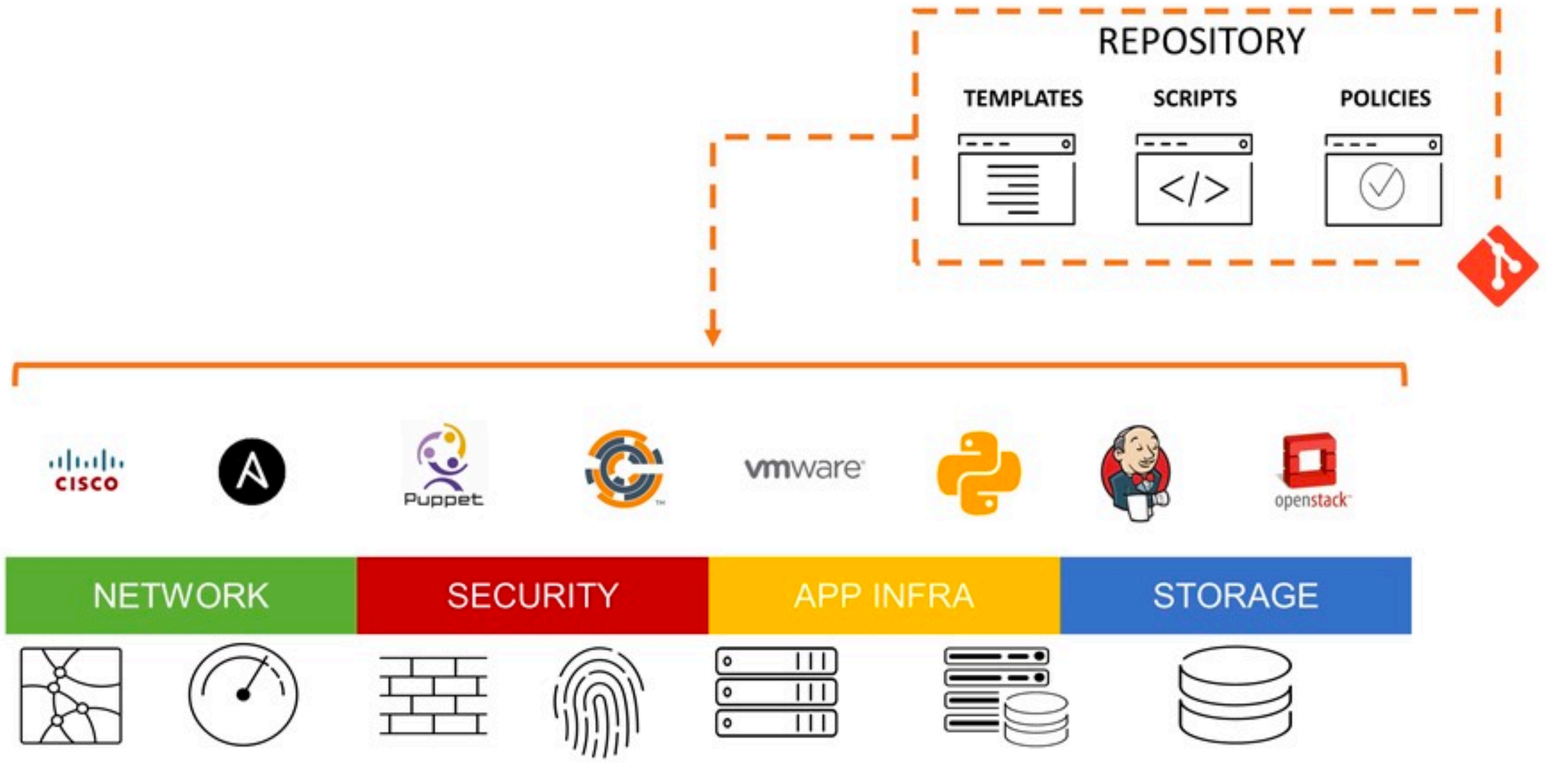
The management of the environment encompasses the management of the lifecycles of each computing resource.”



Infrastructure as Code



Infrastructure as Code Languages



Many formats available:

- Docker Swarm
- Kubernetes
- Ansible
- Terraform
- Chef
- Puppet
- TOSCA

Ansible, Chef, and Puppet

```
- name: Configure webservers
  hosts: webservers
  roles:
    - web
- name: Configure databases
  hosts: dbservers
  roles:
    - db
```

playbook

```
[webservers]
  webs1
[dbservers]
  dbs1
```

inventory

role "web"

```
- name: Install httpd
  yum:  name=httpd  state=latest
  when: ansible_os_family == 'RedHat'

- name: Install apache2
  apt: name=apache2 state=latest
  when: ansible_os_family == 'Debian'
```

role "db"

```
- name: Create database user
  mysql_user: user=myuser password=mypass

- name: Create database
  mysql_db: db=mydb state=present
```

(a)




```
class uniapache {
  if $::osfamily == 'RedHat' {
    $apachename   = 'httpd'
  }
  elsif $::osfamily == 'Debian' {
    $apachename   = 'apache2'
  }
  package { 'apache':
    name => $apachename,
    ensure => 'present',
  }
}
node 'webs1' {
  class { 'uniapache': }
}
node 'dbs1' {
  mysql::db { 'mydb':
    user     => 'myuser',
    password => 'mypass',
    host     => '127.0.0.1'
  }
}
```

(b)




```
node["lamp_stack"]["webs1"].each do
  case node[:platform]
  when 'RedHat'
    apachename = 'httpd'
  when 'Debian'
    apachename = 'apache2'
  end
  package apachename do
    action :install
  end
end

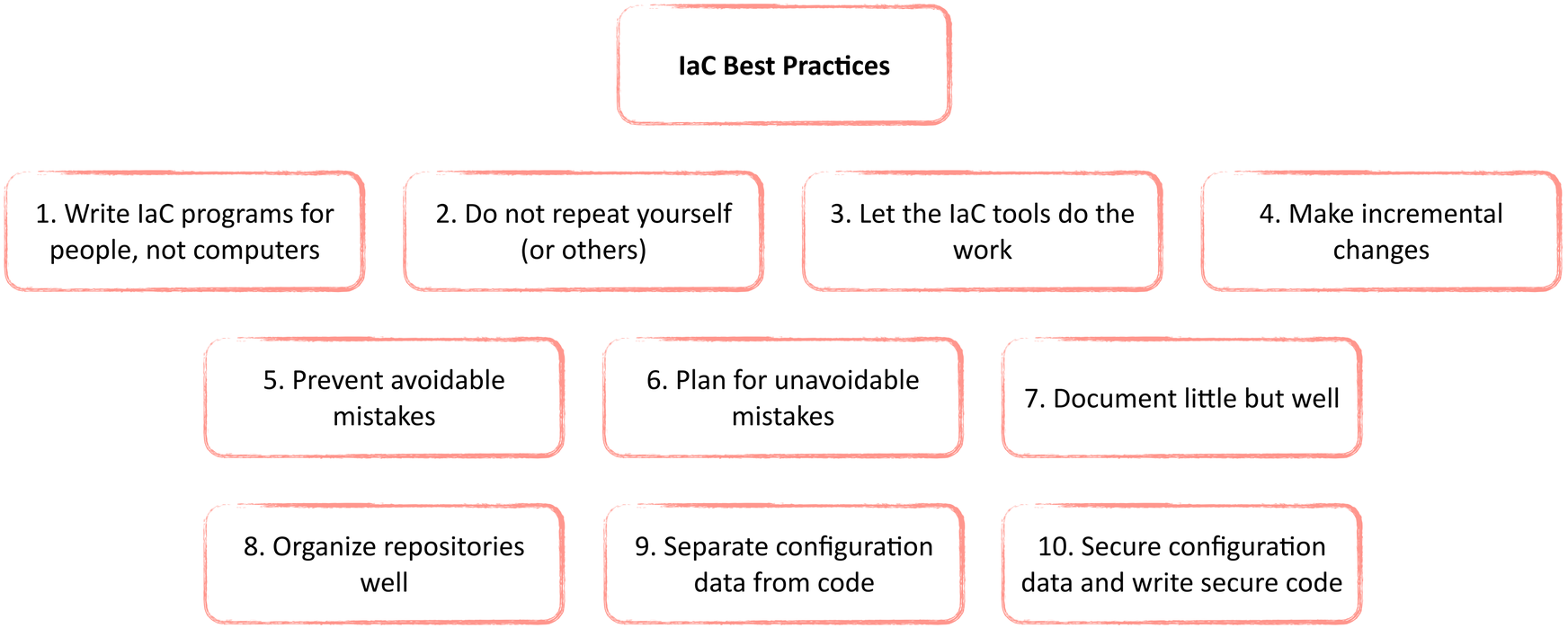
node["lamp_stack"]["dbs1"].each do
  mysql_database 'mydb' do
    connection(
        :host     => '127.0.0.1',
        :username => 'myuser',
        :password => 'mypass'
    )
    action :create
  end
end
```

(c)



Which are the best Practices Adopted by IaC Developers?

10 categories and 33 sub-categories



Which are the best Practices Adopted by IaC Developers?

1. Write IaC programs for people, not computers

- Make names consistent, distinctive, and meaningful (1a).
- Make code style and formatting consistent (1b).
- Make parameters, their types, and defaults explicit (1c).
- Use conditionals properly (1d).

2. Do not repeat yourself (or others)

- Modularize iac programs (2a).
- Reuse code instead of rewriting it (2b).
- Select the right modules for the job and use it correctly (2c).
- Reuse the tools that the community use (2d).

3. Let the IaC tools do the work

- Codify everything (3a).
- Package applications for deployment (3b).
- Do not violate immutability and reproducibility of your infrastructure (3c).
- Do not violate idempotence of iac programs (3d).

4. Make incremental changes

- Use a version control system (4a).
- Favor versionable functionalities (4b).

Which are the best Practices Adopted by IaC Developers?

5. Prevent avoidable mistakes

- Use the correct quoting style (5a).
- Avoid unexpected behaviors whenever possible (5b).
- Use proper values (5c).

6. Plan for unavoidable mistakes

- Write tests as you code (6a).
- Do not ignore errors (6b).
- Use off-the-shelf testing libraries (6c).
- Monitor your environment (6d).

7. Document little but well

- Code as documentation (7a).
- Use document templates (7b).

8. Organize repositories well

- Modularize repositories (8a).
- Use standard folder structures (8b).

9. Separate configuration data from code

- Use configuration datasource (9a).
- Modularize configuration data (9b).
- Select data sources wisely (9c).
- Use configuration templates (9d).

10. Write secure code

- Isolate secrets from code (10a).
- Protect your data at rest (10b).
- Use facts from trusted sources (10c).
- Use standard secure coding practices (10d).

What is TOSCA?

TOSCA is an ***OASIS Standard***

That defines a ***Domain-Specific Language*** (DSL)

For automating the ***Lifecycle Management*** of application, network, and infrastructure services.

TOSCA is an OASIS Open Standard

6 CONTINENTS

70+ COMMUNITIES`

150+ STANDARDS

2000+ PARTICIPANTS

- Global, nonprofit member-driven organization
- Promotes interoperability, innovation, and freedom of choice
- Federation of autonomous working groups
- Broad agenda: cloud computing, cybersecurity, privacy, blockchain, IoT, emergency management, legal…
- Member of European Multi-Stakeholder Platform on ICT Standardization→ EU allows OASIS specs to be referenced in public procurement.
- Liaison relationships offer path to de jure standardization: ISO, IEC (JTC 1), ITU, ETSI
- Home of TOSCA, SAML, PKCS, CAP, OpenDocument, OData, MQTT…

Why is TOSCA the Go-To Choice?

Open Standard

Model-Driven Lifecycle Management

Interoperability-by-design

Domain-Independent

Growing Ecosystem

TOSCA Ecosystem

TOSCA Committee:

170+ people

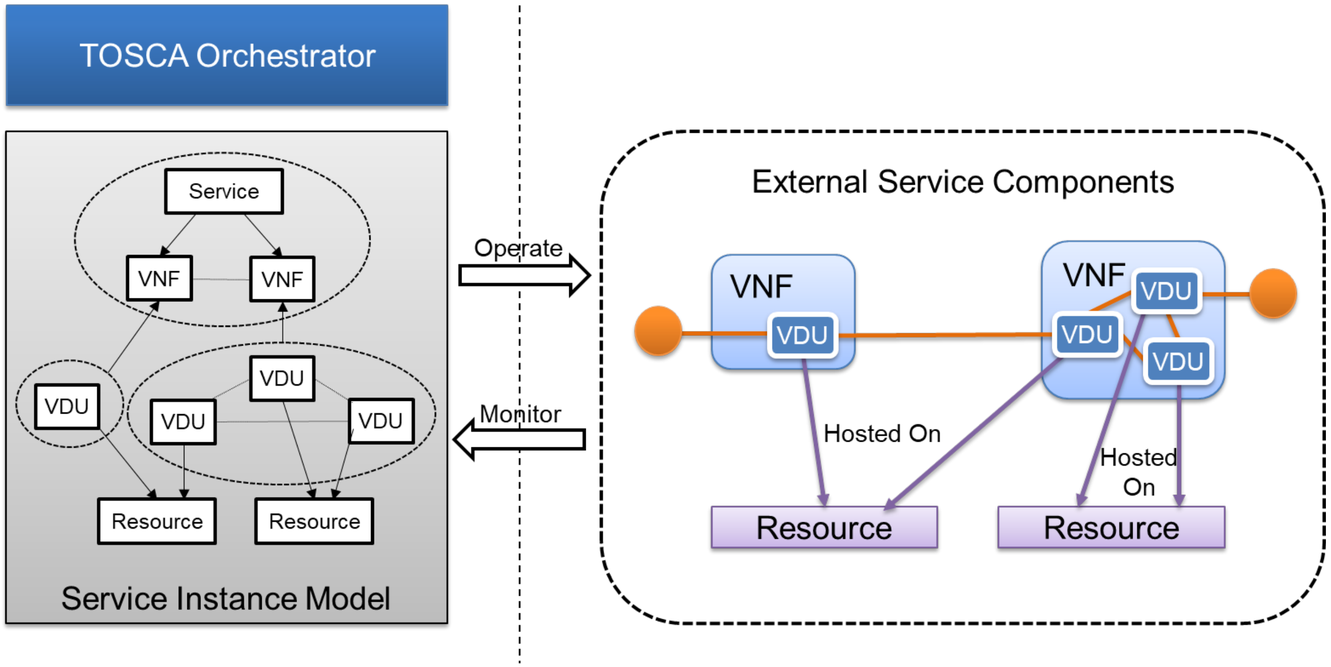
45+ companies/orgs

Associated Companies

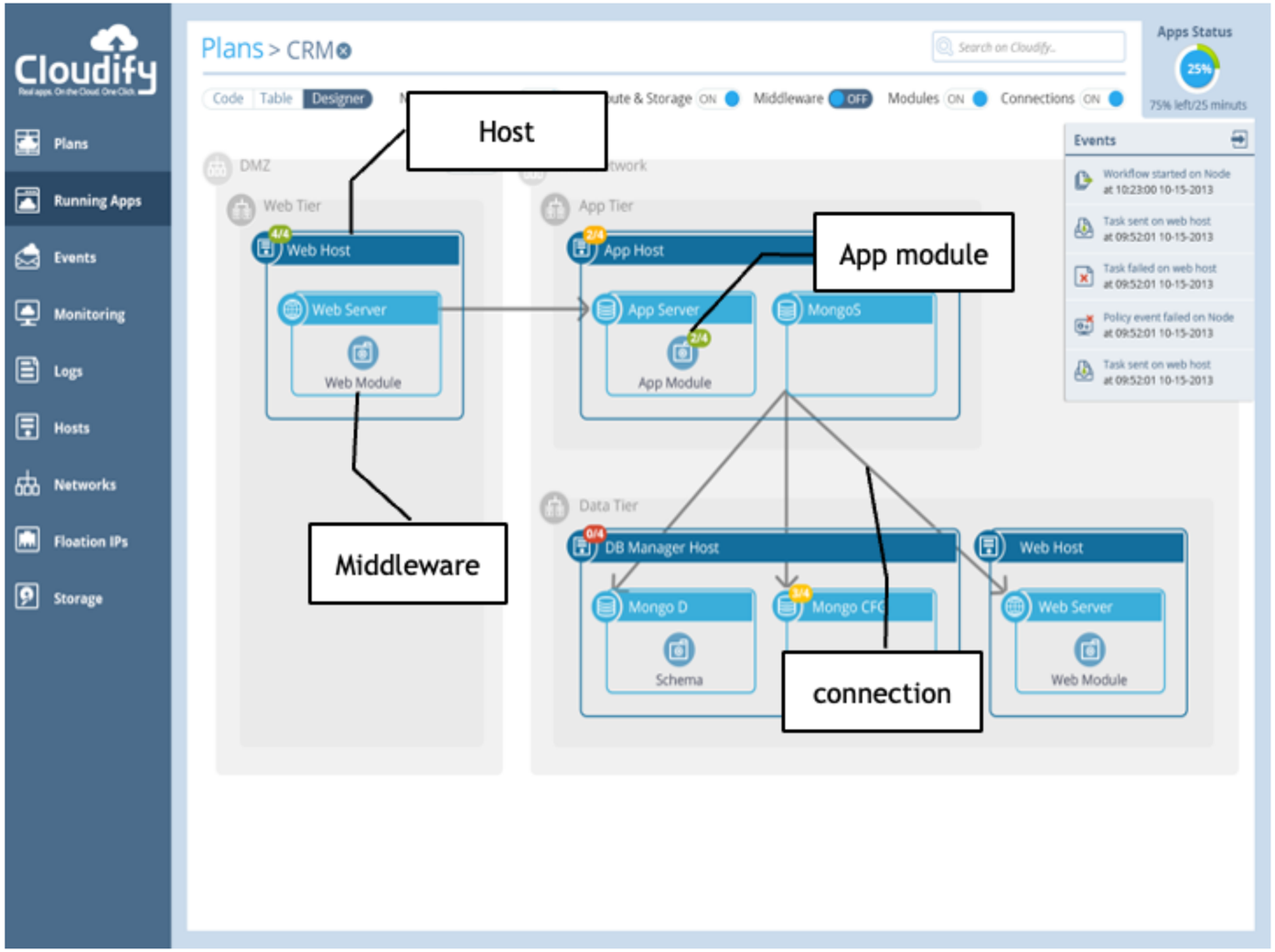


TOSCA uses Model-Driven Lifecycle Management

- TOSCA assumes a model of deployed service instances (***Service Instance Model***)
  - As ***context*** for handling faults and events
  - As ***starting point*** for Moves, Adds, Changes, and Deletions (***MACDs***)



An Application Topology



- Three layers
  - Infrastructure (Cloud or DC objects)
  - Platform or Middleware (App containers)
  - Application modules, schemas and configurations
- Relationships between components:
  - What is hosted on what or installed on what.
  - What is connected to what.

Node Type

Describes a Cloud or Software type (e.g., Server or Apache).
Maps the Type to the actual implementation of the lifecycle interface.

```
tosca.interfaces.node.Lifecycle:
  create:
    description: Basic lifecycle create operation.
  configure:
    description: Basic lifecycle configure operation.
  start:
    description: Basic lifecycle start operation.
  stop:
    description: Basic lifecycle stop operation.
  delete:
    description: Basic lifecycle delete operation.
```

Node Type

Defines properties as YAML maps.
Might defines capabilities (What it can provide to other nodes)

```
tosca.nodes.DBMS
  derived_from: tosca.nodes.SoftwareComponent
  properties:
    dbms_root_password:
      type: string
      description: the root password for the DBMS service
    dbms_port:
      type: integer
      description: the port the DBMS service will listen to for data and requests
  capabilities:
    host:
      type: Container
      containee_types: [ tosca.nodes.Database ]
```

Node Type

Might define requirements (what it needs from other nodes).

```
tosca.nodes.Database:
  derived_from: tosca.nodes.Root
  properties:
    db_user:
      type: string
      description: user account name for DB administration
    db_password:
      type: string
      description: the password for the DB user account
    db_port:
      type: integer
      description: the port the underlying database service will listen to data
    db_name:
      type: string
      description: the logical name of the database
  requirements:
    - host: tosca.nodes.DBMS
  capabilities:
    - database_endpoint: tosca.capabilities.DatabaseEndpoint
```

Relationship Type

Requirements and Capabilities are an implicit way to describe relationships. However, usually you need also explicit ways.
You need hooks to configure the source or target node or both.
So relationships have types and interfaces as well.

- The basic relationship types are:
  - **dependsOn** – abstract type and its sub types:
    - **hostedOn** – a node is contained within another
    - **connectsTo** – a node has a connection configured to another
- The basic interface is configure:
  - **preconfigure_source**, **preconfigure_target**
  - **postconfigure_source**, **postconfigure_target**
  - **add_target**, **remove_target**

Node Templates

An instance of a type (like Object to Class), which has

- specific properties;
- artifacts, specifying what to install and how to install it (mapped to interface hooks);
- requirements and capabilities (or relationships).

```
node_templates:
  wordpress:
    type: tosca.nodes.WebApplication.WordPress
    properties:
      # omitted here for sake of brevity
    requirements:
      - host: apache
      - database: wordpress_db
        interfaces:
          tosca.interfaces.relationships.Configure:
            pre_configure_source: scripts/wp_db_configure.sh
```

```
node_templates:
  wordpress:
    type: tosca.nodes.WebApplication.WordPress
    properties:
      # omitted here for sake of brevity
    requirements:
      - host: apache
      - database: wordpress_db
        relationship_type: my.types.WordpressDbConnection
```

An Application Topology

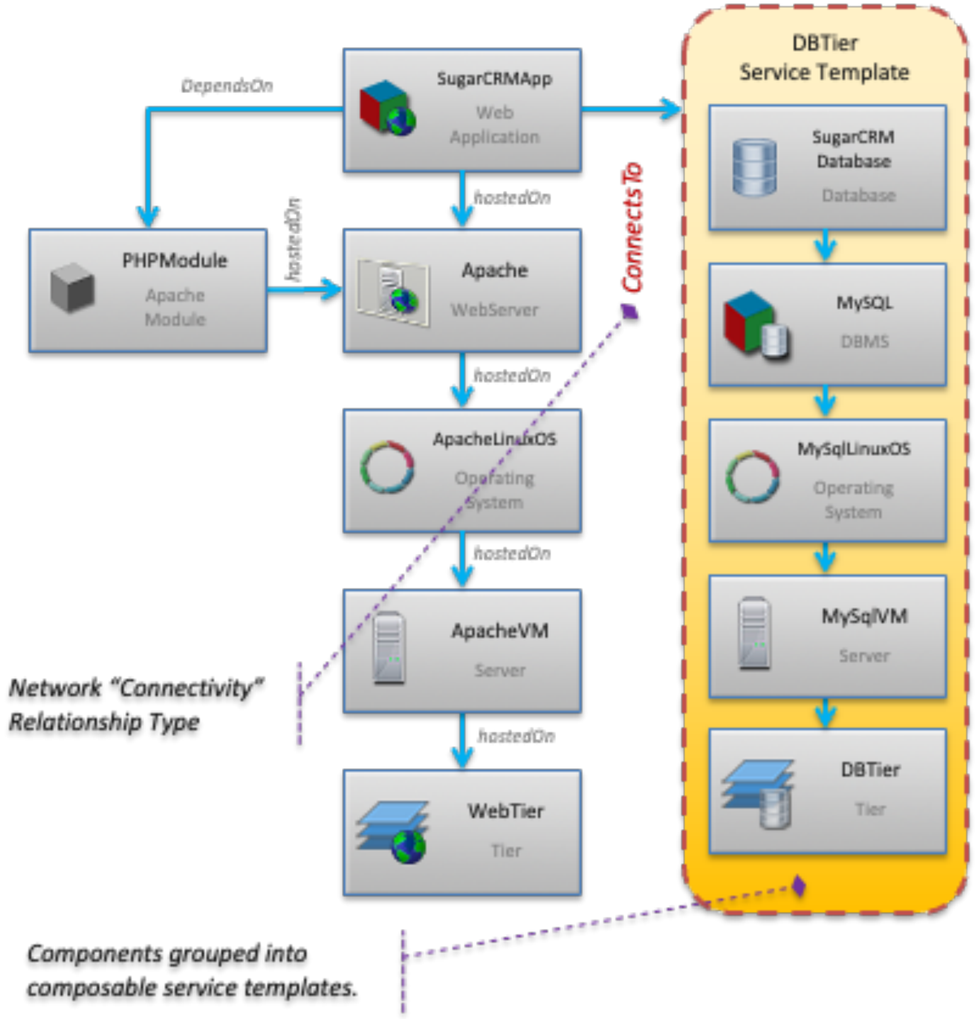
Connectivity

Nodes can "Connect" to other specified node types

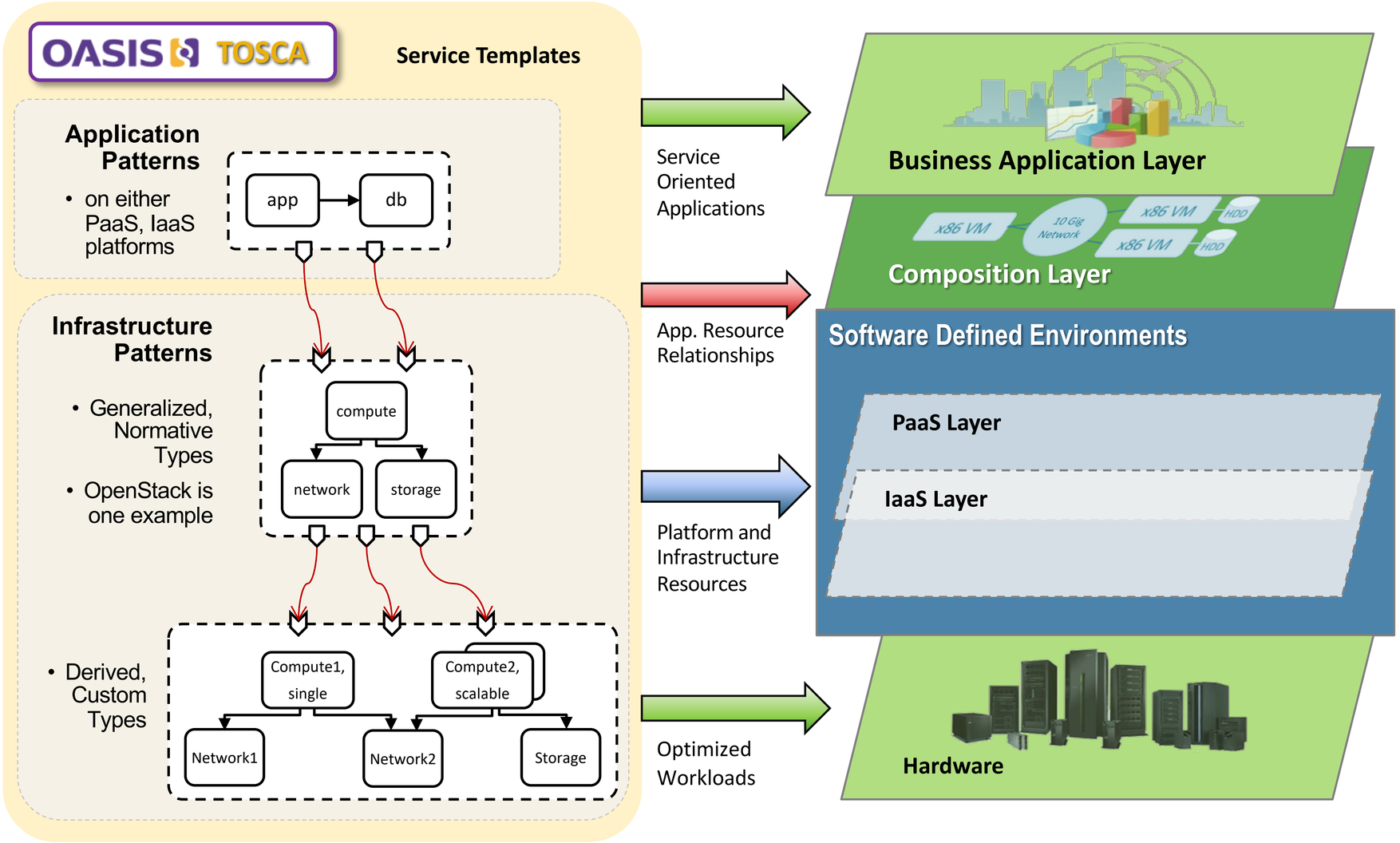
- *Nodes export the types of nodes they require "require" connectivity to,*
- *These are matched to nodes that export they are "capable" of accepting specific connections*

In this example:

- *The SugarCRM Application node "connectsTo" a database node in another "tier"*
- *The "DB Tier" components are packaged into a separate service template permitting Substitution*

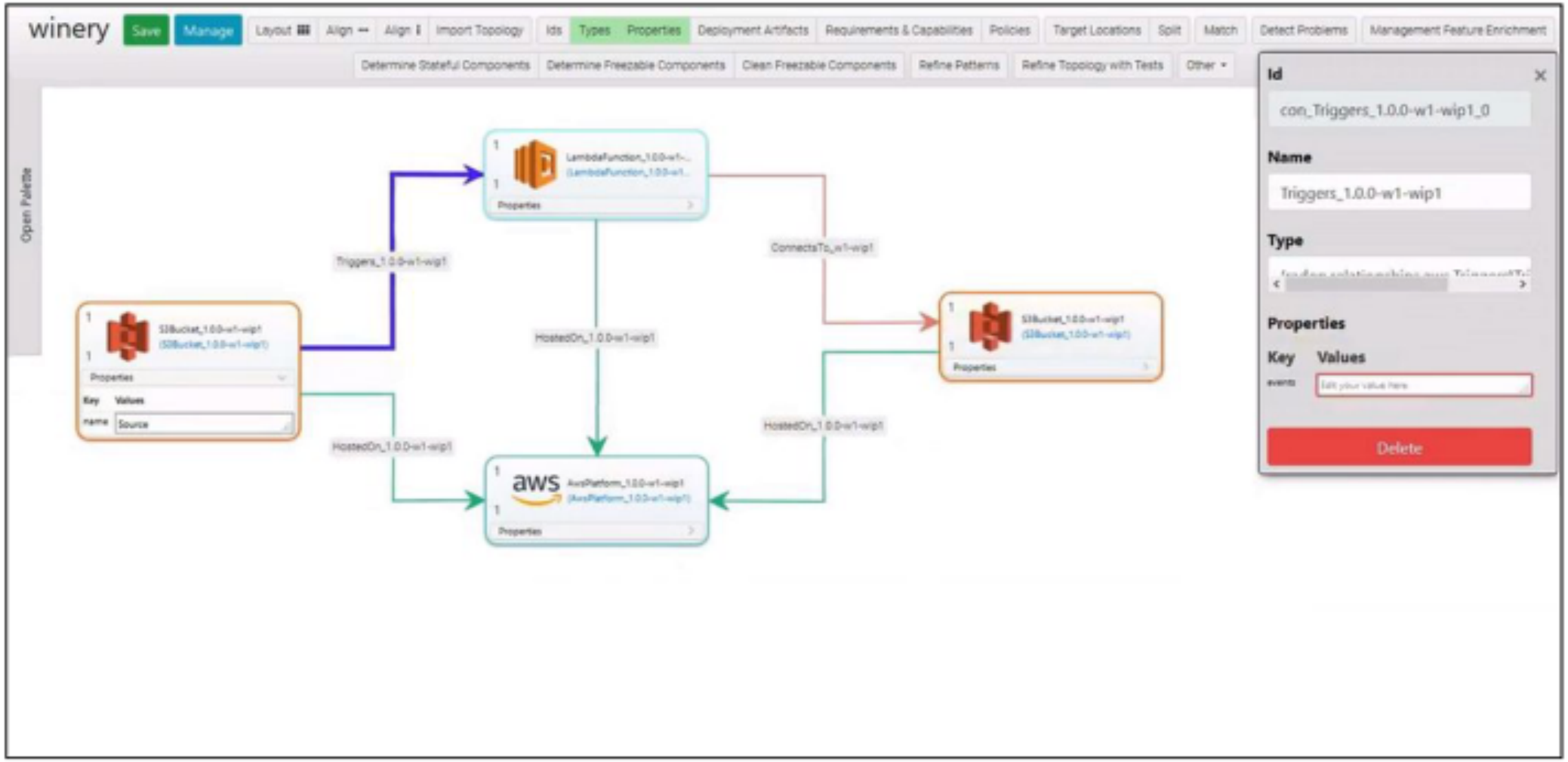


TOSCA service templates can model any cloud application or infrastructure pattern

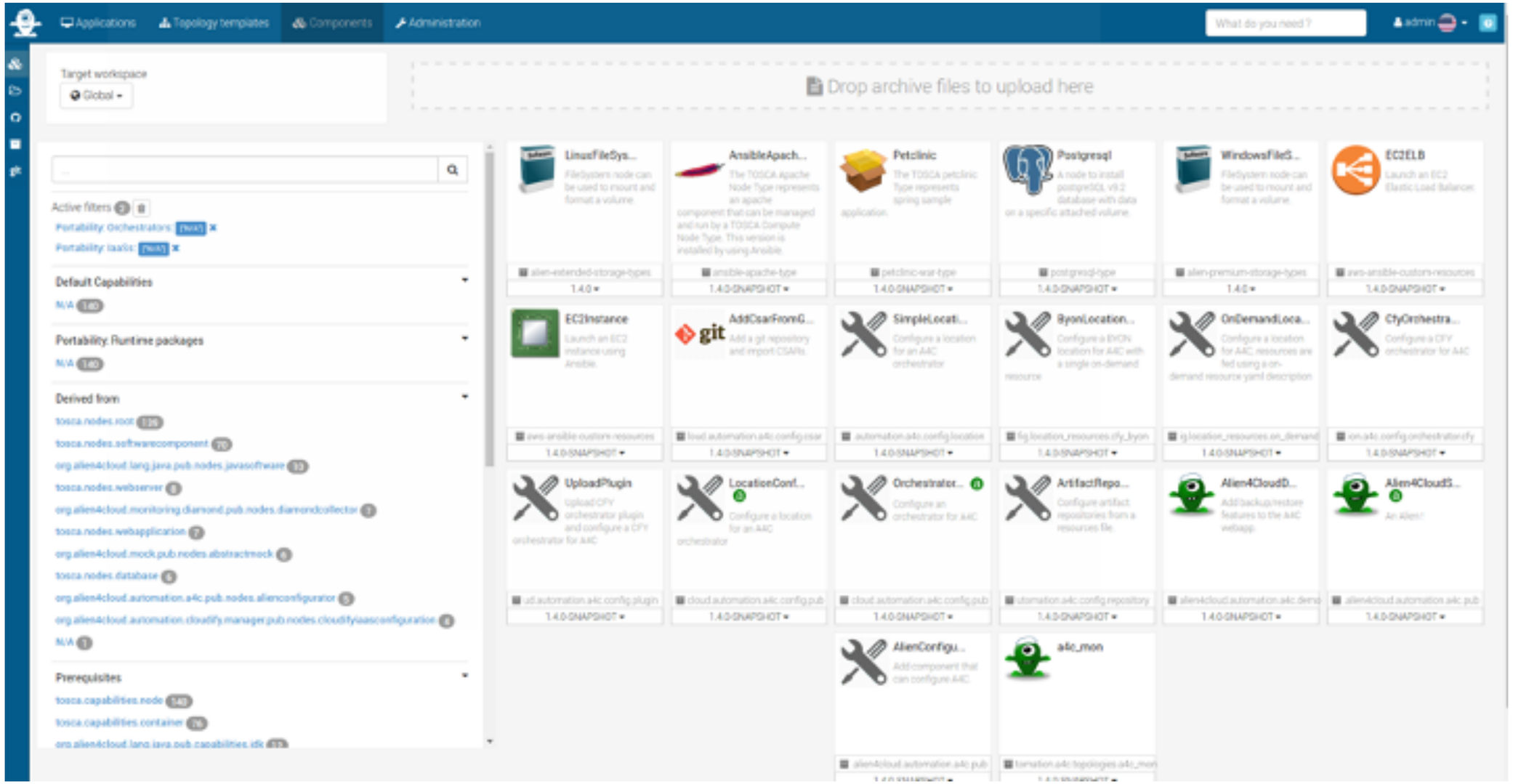


*<u>Value</u>: enables rapid and continuous delivery of diverse set of workloads with **<u>agility</u>** and **<u>optimization</u>** on programmable heterogeneous infrastructure leveraging **<u>reusable</u>** building blocks*

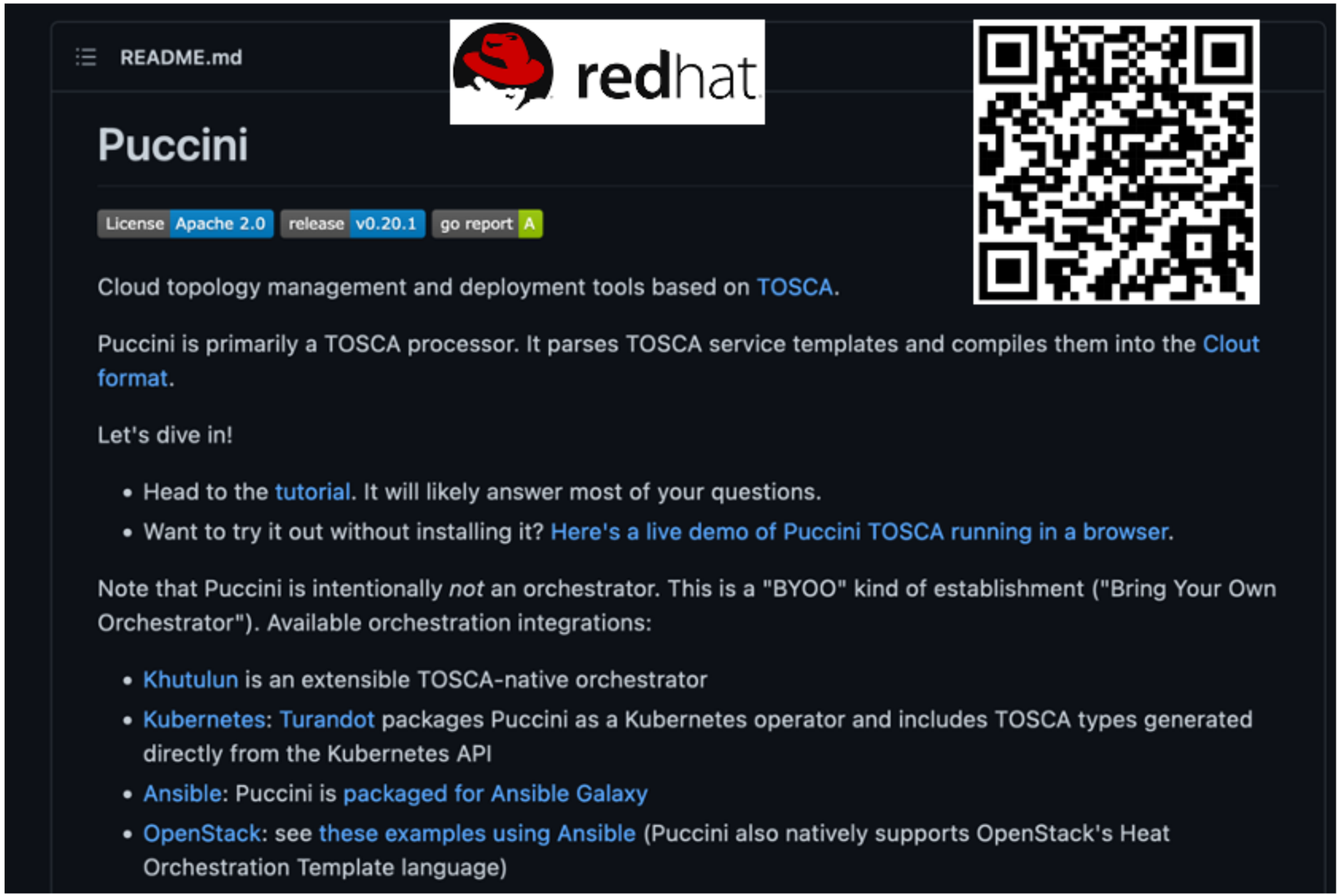
TOSCA Implementations: Eclipse Winery



TOSCA Implementations: Aliens4Cloud



TOSCA Implementations: Puccini




Break